# LitFolio User Manual

A Local-First Research Paper Workbench

v0.3 · May 2026

LitFolio Project

Typeset with LaTeX.

# Contents

**Preface** vi

**I Daily Workflow** 1

**1 Getting Started** 2
1.1 What Is LitFolio . . . 2
1.2 Installation and First Launch . . . 2
1.2.1 Download from Releases (Recommended) . . . 2
1.2.2 Run from Source (Developers) . . . 3
1.3 The Library Directory at a Glance . . . 3
1.4 Next Step: Configure Your First LLM Endpoint . . . 4

**2 The Library** 5
2.1 Main List and Search . . . 5
2.1.1 Search Box . . . 5
2.1.2 FTS5 Full-Text Search Mechanics . . . 5
2.1.3 Sorting and Filtering . . . 6
2.2 Reading States: unread → reading → read → must . . . 6
2.3 Quick Read (TL;DR) . . . 6
2.4 Deep Read: Four-Section Drawer . . . 6
2.5 Title/Abstract Translation . . . 6
2.6 Folders . . . 7
2.7 Tags . . . 7
2.8 Deleting a Paper . . . 7
2.9 Automatic BibTeX Generation . . . 8
2.9.1 BibTeX Key Generation Algorithm . . . 8
2.9.2 Field Mapping . . . 9
2.9.3 Generation on Import . . . 10
2.10 Citation Formatting . . . 10
2.10.1 Four Style Templates . . . 10
2.10.2 Batch Citation Formatting . . . 11
2.11 Batch Citation Export . . . 11
2.11.1 BibTeX (.bib) . . . 11
2.11.2 RIS (.ris) . . . 11
2.11.3 Formatted Text . . . 12
2.11.4 File Naming and Trigger . . . 12
2.12 Smart Collections . . . 12
2.12.1 Rule Structure . . . 13
2.12.2 SQL Translation Engine . . . 13
2.12.3 Real-Time Updates . . . 13
2.12.4 Performance . . . 13
2.13 Reading Queue . . . 14

2.13.1 Queue Data Model . . . 14
2.13.2 Drag-and-Drop Sorting . . . 15
2.13.3 Target Dates and Overdue Alerts . . . 15
2.13.4 Automatic Status Update . . . 15
2.13.5 Queue vs. Folders/Tags . . . 15
2.14 Paper Deduplication . . . 15
2.14.1 Three-Level Matching Strategy . . . 15
2.14.2 Title Preprocessing . . . 16
2.14.3 Full-Library Scan . . . 16
2.14.4 Merge Workflow . . . 16

**3 Importing Papers** **18**
3.1 arXiv ID / DOI . . . 18
3.2 Local PDF Files . . . 19
3.2.1 PDF Metadata Extraction Strategy . . . 19
3.3 Semantic Scholar Search . . . 20
3.4 Batch Folder Import . . . 20
3.4.1 Recursive Scanning . . . 21
3.4.2 Filename Heuristic Parsing . . . 21
3.4.3 Concurrent Processing and Progress Tracking . . . 21
3.5 Redirect from RSS / arXiv Browse . . . 21

**4 The Reader** **22**
4.1 Three-Pane Layout . . . 22
4.1.1 PDF Text Extraction and Caching . . . 22
4.2 Highlights: Text Selection + Region . . . 23
4.3 Selection Translation . . . 23
4.4 Term Table and Term Network . . . 24
4.4.1 Term Extraction Algorithm . . . 24
4.5 Structured Note Cards . . . 26
4.5.1 Default Sections . . . 26
4.5.2 AI Auto-Fill Mechanism . . . 27
4.5.3 Sections and Full-Text Search . . . 27
4.6 Annotation Color Semantics . . . 27
4.6.1 Workflow . . . 28
4.6.2 Custom Labels . . . 28
4.7 Markdown Notes . . . 28
4.8 PDF Search . . . 28
4.9 Page Shortcuts . . . 29

**II Discovery and Synthesis** **30**

**5 arXiv Browse** **31**
5.1 Category Tree . . . 31
5.2 Pagination and ”Exhausted” . . . 31
5.3 Importing . . . 32

**6 RSS Feeds** **33**
6.1 Feed Management . . . 33
6.2 Conditional GET Refresh . . . 33
6.3 Entry Details and Translation . . . 34
6.4 Direct Import from Feed Items . . . 34

**7 Topic Discovery** **35**
7.1 Two-Tab Mental Model . . . 35
7.2 Search Recall . . . 35
7.2.1 Query Expansion . . . 36
7.2.2 Query Expansion Algorithm Details . . . 36
7.2.3 Time Window and Sorting . . . 36
7.3 Survey Generation . . . 36
7.3.1 Pipeline . . . 36
7.3.2 Saving and Restoring . . . 38
7.4 Topic Alerts . . . 39
7.4.1 Creating an Alert . . . 39
7.4.2 Alert Execution Flow . . . 39
7.4.3 Auto Import . . . 40
7.4.4 Management and Viewing . . . 40

**8 Library Q&A (RAG)** **41**
8.1 How It Works . . . 41
8.2 Multi-Turn Chat Interface . . . 42
8.2.1 Conversation State Model . . . 43
8.2.2 Multi-Turn RAG Flow . . . 43
8.2.3 Conversation History Passing Details . . . 43
8.2.4 UI Interaction . . . 44
8.3 Save as Knowledge Note . . . 44

**9 Knowledge Graph** **45**
9.1 Network Graph . . . 45
9.2 Mind Map . . . 46
9.3 Manual Relationship Creation . . . 46
9.4 Citation Network . . . 47
9.4.1 Data Fetching Flow . . . 47
9.4.2 Tree Display . . . 48
9.4.3 Relationship with the Knowledge Graph . . . 48
9.5 Concept Graph . . . 48
9.5.1 Concept Extraction Pipeline . . . 48
9.5.2 Concept Relationship Types . . . 49
9.5.3 Graph Visualization Integration . . . 50
9.5.4 Manual Management . . . 50
9.6 AI-Discovered Relationships . . . 50
9.7 Filtering and Legend . . . 51
9.8 Reader Integration . . . 51

**10 Similar Paper Recommendations** **52**
10.1 Usage . . . 52
10.1.1 Recommendation Fetching Flow . . . 52
10.1.2 Result Display . . . 53
10.1.3 Edge Cases . . . 53

**11 Literature Review Draft Generation** **54**
11.1 Usage . . . 54
11.1.1 Generation Pipeline . . . 54
11.1.2 Input Materials . . . 55
11.2 Output and Editing . . . 56

**III Advanced Tools and Administration** **57**

**12 Multi-paper Comparison** **58**
12.1 How to Use . . . 58
12.1.1 Comparison Generation Pipeline . . . 58
12.1.2 Comparison Table Structure . . . 59
12.1.3 Editing and Saving . . . 59

**13 Markdown Export** **60**
13.1 Export Content . . . 60
13.1.1 File Structure . . . 60
13.1.2 Frontmatter Fields . . . 61
13.1.3 Highlights Grouped by Label . . . 61
13.1.4 Terms and Bidirectional Links . . . 61
13.2 Incremental Export Algorithm . . . 61
13.3 Configuration and Triggers . . . 62

**14 Command Palette** **63**
14.1 Three Modes . . . 63
14.1.1 Fuzzy Matching Algorithm . . . 64
14.1.2 Result Grouping and Display . . . 64

**15 Custom Metadata Fields** **65**
15.1 Field Management . . . 65
15.1.1 Field Types . . . 65
15.1.2 Data Model . . . 65
15.2 Usage . . . 65

**16 Settings** **67**
16.1 LLM Configuration . . . 67
16.1.1 Pulling Available Models . . . 67
16.1.2 Testing the Connection . . . 68
16.2 Task Assignment . . . 68
16.2.1 LLM Call Details per Task . . . 68
16.3 WebDAV Sync . . . 69
16.4 Export Settings . . . 70
16.5 Annotation Color Settings . . . 70
16.6 Custom Field Settings . . . 70
16.7 Topic Alert Settings . . . 70

**17 Data and File Layout** **72**
17.1 Top-Level Structure . . . 72
17.2 library.db Table Overview . . . 72
17.2.1 Core . . . 72
17.2.2 Discovery and Reading . . . 73
17.2.3 Notes and Knowledge . . . 73
17.2.4 Configuration and Indexing . . . 73
17.3 papers/<id>/ Per-Paper Layout . . . 73
17.4 Performance Optimization . . . 74
17.4.1 Virtual Scrolling . . . 74
17.4.2 Embedding Cache . . . 74
17.5 Backups . . . 74

**18 Keyboard Shortcuts Quick Reference** **76**
18.1 Reader (`/reader`) . . . 76
18.2 Global . . . 76

**19 Troubleshooting** **77**
19.1 LLM Call Failure . . . 77
19.2 PDF Load Failure . . . 77
19.3 RSS Not Fetching / Always “Already Up to Date” . . . 77
19.4 WebDAV Sync Failure . . . 78
19.5 Database Locked . . . 78
19.6 Partial Batch Import Failure . . . 78
19.7 Concept Extraction Returns Empty . . . 78
19.8 Citation Network / Similar Papers Empty . . . 79
19.9 Smart Collection Not Updating . . . 79

**A LLM Endpoint Compatibility List** **80**

**B arXiv Main Category Quick Reference** **81**

**C Recommended RSS Feeds** **82**

**D License** **83**

# Preface

## About This Manual

LitFolio is a local-first desktop workbench for research papers. It brings together arXiv browsing, PDF reading, highlight annotations, term extraction, multi-turn RAG conversation, knowledge graphs, topic survey generation, literature comparison, citation management, and WebDAV synchronization—all inside a single Tauri application, with every piece of data stored under the `~/Litera-Library/` directory on your own machine.

This manual is written for three audiences:

- **New users**—if you have never used LitFolio, start with Chapters 1–4. You will go from importing an arXiv paper through deep reading, translation, and the term network in about thirty minutes.
- **Active researchers**—if you want to master topic surveys, multi-turn RAG dialogue, knowledge graphs, WebDAV sync, and other advanced capabilities, continue with Part II, Part III, and Chapter 12.
- **Power users**—who need literature comparison, survey drafting, BibTeX management, smart collections, the command palette, and other productivity tools. Focus on Part III (Chapters 11–17).

## What You Will Learn

1. Three import pathways (arXiv/DOI, local PDF, Semantic Scholar search), batch folder import, and automatic deduplication.
2. The full reading workflow: list → TL;DR quick read → four-section deep read → translation → open in PDF reader → highlights (with semantic color labels) → structured note cards → term network.
3. Automatic BibTeX generation and citation formatting (APA/IEEE/GB/T 7714/Chicago), with batch export to `.bib` or `.ris` files.
4. Synthesizing scattered papers into a topic survey, or asking questions like "Which papers discuss limitations of CPA?" and getting answers with numbered citations—including multi-turn follow-up conversation.
5. Knowledge graphs: paper-relationship networks, citation networks, and concept maps, with AI-powered discovery of method evolution and cross-paper connections.
6. Advanced tools: structured multi-paper comparison, literature review draft generation, similar-paper recommendations, reading queues, smart collections, custom metadata fields, Markdown/Obsidian export, and the command palette (`Ctrl+K`).
7. Configuring LLM endpoints, assigning different tasks to different models, and synchronizing your entire library across machines via WebDAV.
8. The physical layout of data on disk, making it easy to back up, script, or troubleshoot.

## Typographic Conventions

**Violet** is used for chapter headings and emphasis, matching the primary interactive color in the product (DESIGN token `violet`).

`Ctrl+F` Keyboard shortcuts appear in rounded chips.

`file.tsx` File paths and code identifiers use monospace.

`pnpm tauri dev` Terminal commands likewise use monospace.

Two recurring callout boxes appear throughout the manual:

> **Tip** Light-violet box: supplementary notes and quick-use tips. Safe to skip.

> **Warning** Amber box: operations that are irreversible, overwrite data, or require careful understanding before proceeding. Please read these thoroughly.

## Feedback and Contributions

LitFolio is released under GPL-3.0-or-later. For bug reports, feature suggestions, and patches, please open an issue or pull request on the project repository. This manual lives in the same repository as the main codebase; corrections and additional use cases are always welcome.

# Part I

# Daily Workflow

Chapter 1

# Getting Started

## 1.1 What Is LitFolio

LitFolio is a desktop research paper workbench built on a **local-first** principle: all data—metadata, PDFs, notes, highlights, terms, surveys—lives directly under `~/Litera-Library/` on your own machine. Network access is only needed to fetch arXiv metadata, parse RSS feeds, and call LLMs. In other words, you can read, write, and search even when offline; moving to another computer is as simple as syncing that directory.

LitFolio is composed of eight major feature areas:

1. **Library**—local paper list, reading-state machine, TL;DR, deep read, translation, folders, tags, automatic BibTeX generation, citation formatting, batch export.
2. **Reader**—three-pane PDF view with a highlight list on the left and notes / selection translation / terms tabs on the right; structured note cards and semantic annotation colors.
3. **Import**—three entry points: arXiv/DOI metadata, local PDF files, and Semantic Scholar search; batch folder import with deduplication.
4. **Discovery**—arXiv category browsing, RSS subscriptions, topic survey generation, and topic alerts.
5. **Ask**—multi-turn RAG conversation over your read papers: query rewriting, multi-path FTS5 retrieval, answers with citation numbers, and context-aware follow-ups.
6. **Knowledge Graph**—paper-relationship network, citation network, and concept map, with AI-powered discovery of connections and method evolution.
7. **Advanced Tools**—multi-paper comparison, literature review draft, similar-paper recommendations, reading queue, smart collections, custom metadata fields, Markdown export, command palette.
8. **Settings**—multiple LLM endpoint configurations, per-task model assignment, whole-library WebDAV sync.

## 1.2 Installation and First Launch

LitFolio is built with Tauri 2 + React 18 and is available through two channels:

### 1.2.1 Download from Releases (Recommended)

Go to the project repository's Releases page and download the installer for your platform:

- Linux: `LitFolio_x.y.z_amd64.AppImage` or `.deb`
- macOS: `LitFolio_x.y.z_universal.dmg`
- Windows: `LitFolio_x.y.z_x64-setup.exe`

### 1.2.2 Run from Source (Developers)

1. Install `pnpm`, `rustup` (with the stable toolchain), and the platform-specific Tauri system dependencies (on Linux typically `webkit2gtk-4.1`, `librsvg`, `libsoup-3.0`).
2. Run `pnpm install` to install frontend dependencies.
3. Run `pnpm tauri dev` to start development mode; the first launch will compile Rust crates and may take a few minutes.

> **Tip** In development mode the window title includes a "[dev]" badge. The database is the same as the production build (both live under `~/Litera-Library/`), so papers you import, deep-read, or annotate in dev mode will also appear in the packaged build.

## 1.3 The Library Directory at a Glance

On first launch, LitFolio automatically creates `~/Litera-Library/` and initializes an empty database. You will immediately see the left navigation bar (**Library** / **Import** / **Browse** / **Feeds** / **Topics** / **Ask** / **Settings**) and an empty main list.

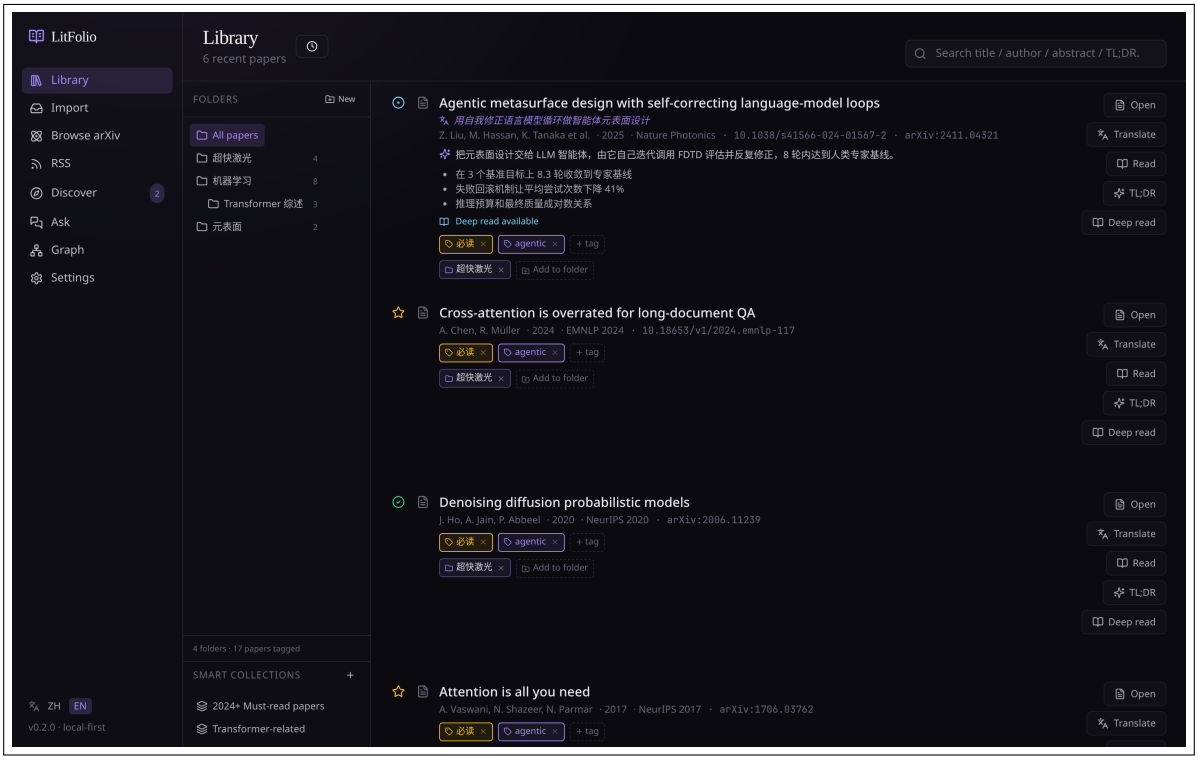


**Figure 1.1** LitFolio main interface. Left: navigation bar; center: folder sidebar; right: paper list with search box. Deep violet (`violet`) is the primary interactive color, used for buttons, links, and emphasis.

The physical layout of the library directory (detailed in Chapter 12) is roughly:

```
~/Litera-Library/
library.db                                  // SQLite main database
papers/<paperId>/                           // one directory per paper
paper.pdf
meta.json
```

```
note.md
pdf-text.txt
backups/                                        // automatic backups
sync/                                             // sync snapshots
```

## 1.4 Next Step: Configure Your First LLM Endpoint

LitFolio does not ship with any built-in LLM. To use TL;DR, deep reading, translation, term extraction, topic surveys, and library Q&A, you must first add at least one endpoint under *Settings* → *LLM Configuration* (any OpenAI-compatible API or a local Ollama instance will work). See **Chapter 15** for full details; you can jump there now and come back, or proceed to Chapter 2 to explore the features that do not require an LLM.

Chapter 2

# The Library

The library page (`/library` route) is the screen you will spend the most time on. Its design goal is simple: one glance at the list should tell you what these papers are about, which ones deserve a deep read, and which are just RSS items you have not touched yet.

## 2.1 Main List and Search

Each row in the main list displays: title, authors, year/journal, reading-status badge, TL;DR summary (if generated), key-finding chips, and tag chips. Clicking anywhere on a row opens the detail drawer on the right (see Section 2.4).

### 2.1.1 Search Box

The search box at the top uses SQLite FTS5: titles, abstracts, TL;DRs, notes, and highlights are all indexed in a unified full-text index. Keywords are space-delimited with AND semantics. To search for an exact phrase, wrap it in double quotes.

### 2.1.2 FTS5 Full-Text Search Mechanics

LitFolio's full-text index is based on the SQLite FTS5 virtual table `papers_fts`, using the `unicode61` tokenizer. The table indexes five fields: title, authors, abstract, TL;DR, and notes. INSERT / UPDATE / DELETE triggers keep it in sync with the `papers` main table—papers are automatically re-indexed on import or update.

The search pipeline:

1. **Input sanitization**—the `escape_fts` function splits user input on spaces, strips FTS5 special characters (`"():.-`), wraps each token as `"token"*` (prefix matching), and joins them with AND. For example, `retrieval augmented` becomes `"retrieval"* AND "augmented"*`.
2. **BM25 ranking**—FTS5's built-in BM25 algorithm computes a relevance score automatically. Results are sorted by ascending BM25 (lower is more relevant), with ties broken by descending import time.
3. **Empty-query fallback**—if sanitization produces an empty query (e.g., the user typed only special characters), the system falls back to `list_recent`, returning papers in reverse chronological order.

> **Tip** The FTS5 `unicode61` tokenizer splits CJK text character by character, so searching for a Chinese phrase like "neural network" is equivalent to searching for each character individually with AND—short characters may bring in noise. For precise CJK searching, consider using English keywords or the English names of technical terms.

### 2.1.3 Sorting and Filtering

The sidebar's Must-read / Reading / Read / Unread filters correspond to the `must`/`reading`/`read`/`unread` status values. Clicking a folder (see 2.6) or a tag chip applies the corresponding filter.

## 2.2 Reading States: unread → reading → read → must

Each paper has a **reading state**, initially `unread`:

**unread** Not yet touched. Papers imported via RSS or arXiv default to this state.

**reading** Currently being read. Signals that you want this paper to appear near the top of the list.

**read** Finished; the paper can fade into the background.

**must** "Must read"—a higher-priority marker than `read`, also given priority when generating topic surveys.

Clicking the status badge cycles through the states in order. This four-state mechanism is a deliberate design choice: LitFolio does not assume that reading a paper once means you are done with it—it preserves the semantics of "important papers deserve to be revisited."

## 2.3 Quick Read (TL;DR)

The main list has a "Quick Read" button on the right. Clicking it invokes the "TL;DR generation" task (bound by default to your configured `tldr` model). The returned paragraph is saved directly to the database as a compact summary and appears instantly the next time you open the paper.

> **Tip** Once generated, the TL;DR is cached in SQLite. Clicking the same paper again will not re-invoke the LLM. To force regeneration, click "Regenerate" in the drawer.

## 2.4 Deep Read: Four-Section Drawer

"Deep Read" is one of the core interactions that sets LitFolio apart from other reader tools. Click the "Deep Read" button on a paper row and the right drawer presents content in four structured sections:

The four-section design lets you judge whether a paper deserves a detailed read without going through the entire text. The **Differences** and **Limitations** sections alone are usually enough to decide whether to open the PDF. The full result is likewise cached.

## 2.5 Title/Abstract Translation

The drawer includes a "Translate" button that calls the "translation" task (bound to your `translate` model). The result is presented as a bilingual side-by-side view, and the translated title and abstract are saved in the metadata for quick scanning.

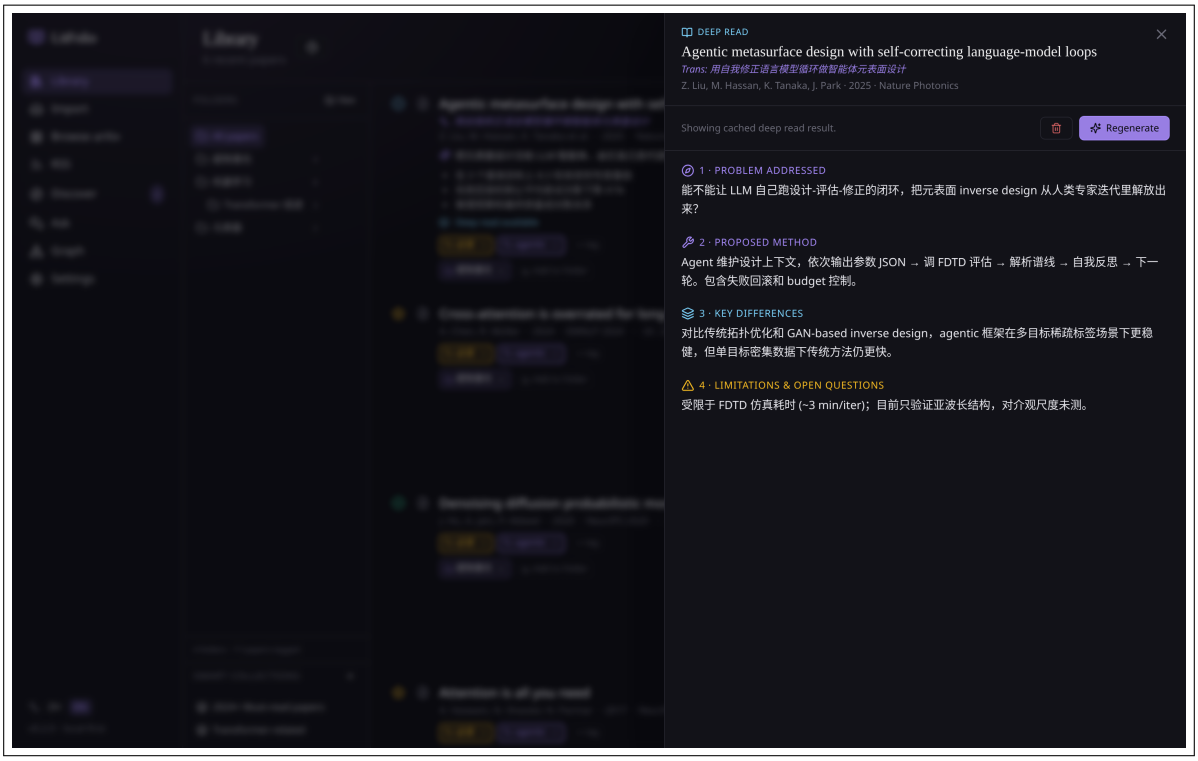


**Figure 2.1** Deep-read drawer. Top to bottom: **What problem does it solve**, **What method is proposed**, **How does it differ from others**, **Limitations and open issues**—each section is extracted from the full text and has a `Regenerate` button beside it.

> **Tip** Different tasks can be bound to different models—for instance, you can assign TL;DR to DeepSeek and translation to a local Ollama instance. See **Chapter 15**.

## 2.6 Folders

Folders are hierarchical (unlike tags) and support multiple membership—a single paper can reside in both "ML/Transformer" and "Applications/NLP" simultaneously.

To create a folder: right-click an empty area in the sidebar → `New`. To manage membership: drag and drop, or click "Move to…" in the drawer.

## 2.7 Tags

Tags are flat, string-based labels created through the input box in the drawer. A paper can have any number of tags. The distinction from folders:

- Folders = **research domain taxonomy**, relatively stable, 2–3 levels deep.
- Tags = **ad-hoc topic/project markers**, ephemeral and flat.

## 2.8 Deleting a Paper

The drawer has a "Delete" button at the bottom. Deleting removes: the entire `papers/<id>/` directory, the SQLite row, and all associated highlights, notes, and terms.

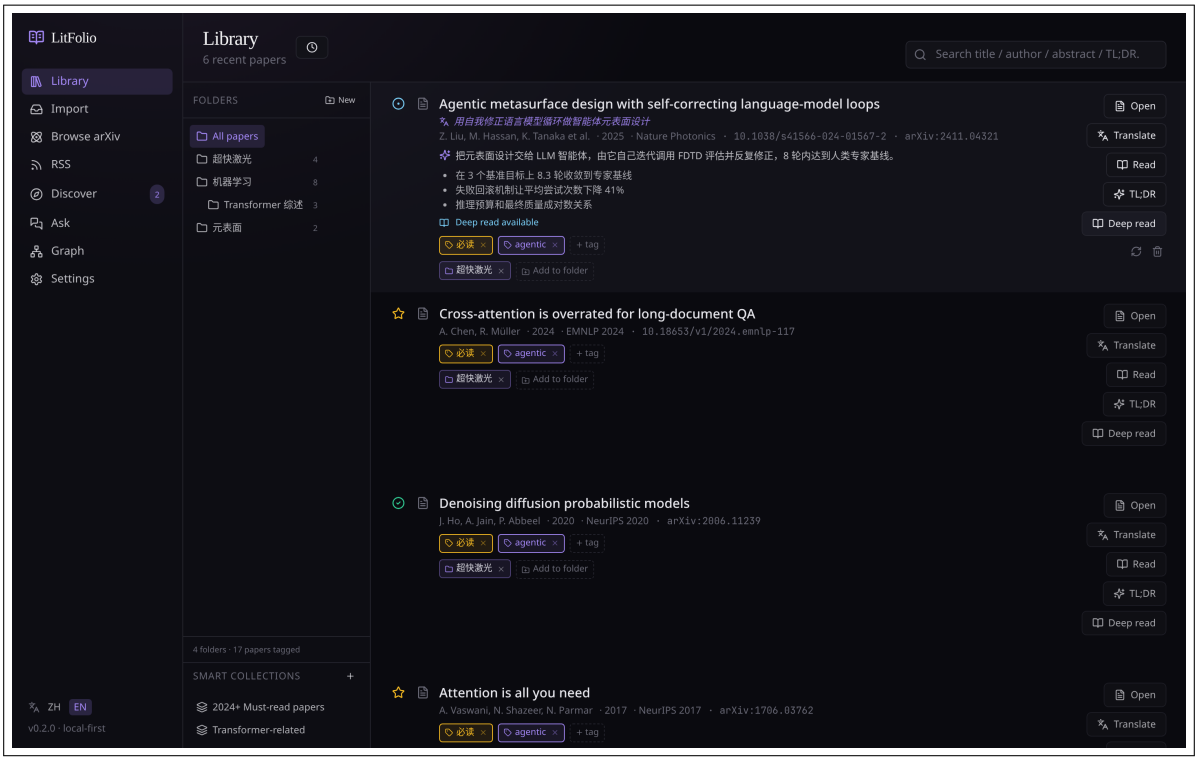


**Figure 2.2** Sidebar folders. Selecting a folder filters the main list to its contents; combined with the search box, this gives you "full-text search within a folder."

> **Warning** Deletion is irreversible in the database, but LitFolio moves `papers/<id>/` to `backups/deleted/<timestamp>/` instead of running `rm -rf`. If you accidentally delete something, you can recover it from `backups/deleted/`.

## 2.9 Automatic BibTeX Generation

When a paper is imported, LitFolio automatically generates a BibTeX entry from its metadata and stores it in the `bibtex` database field. Both the drawer and the reader header have a "Copy BibTeX" button that copies the entry to the system clipboard.

### 2.9.1 BibTeX Key Generation Algorithm

The BibTeX key is a paper's unique citation identifier, formatted as `<first-author-surname-lowercase><y` e.g. `vaswani2017attention`. The generation pipeline:

1. **Extract first author's surname**—take the first element of the `authors` array, split on space or comma, take the last token (to handle both `Firstname Lastname` and `Lastname, Firstname` formats), lowercase it, and strip non-ASCII characters (keeping only `a-z` and hyphens). If the author list is empty, use `unknown`.
2. **Extract first significant title word**—split the title on spaces, skip stop words (`a/an/the/on/in/of/f` take the first non-stop word, lowercase it, and keep only `a-z0-9`. If the title consists entirely of stop words (extremely rare), take the first word.
3. **Concatenate**—`<surname><year><first-word>`. If the year is missing, use `0000`.

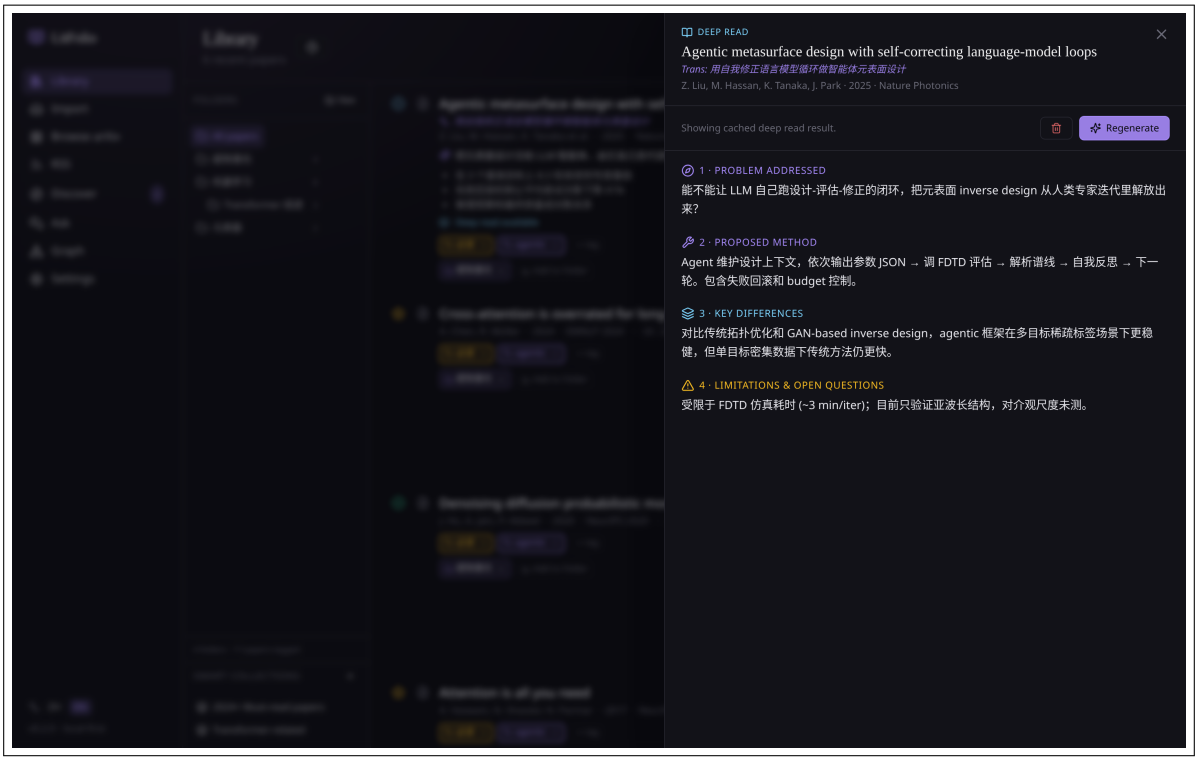


**Figure 2.3** BibTeX section in the paper drawer. One click copies the entry to the clipboard, ready to paste into a LaTeX document.

4. **Deduplicate**—after generation, check whether the key already exists in the library. If so, append `a`, `b`...until the key is unique. This ensures that importing the same paper from different sources will never produce a BibTeX key collision.

### 2.9.2 Field Mapping

| BibTeX Field | Source | Processing Rule |
|---|---|---|
| `title` | `papers.title` | Used directly |
| `author` | `papers.authors` | JSON array joined with `and` |
| `year` | `papers.year` | Used directly; omitted if missing |
| `journal/booktitle` | `papers.venue` | Written if venue exists; omitted otherwise |
| `doi` | `papers.doi` | Written if DOI exists; omitted otherwise |
| `eprint` | `papers.arxiv_id` | Written if arXiv ID exists |
| `archivePrefix` | Hardcoded | Fixed to `arXiv` when arXiv ID exists |
| `primaryClass` | arXiv ID parsing | Inferred from the arXiv ID prefix (e.g. `cs` → `cs.AI`); omitted if uninferrable |

| BibTeX Field | Source | Processing Rule |
|---|---|---|
| `url` | Composed | If DOI exists → `https://doi.org/<doi>`; if arXiv ID exists → `https://arxiv.org/abs/<id>`; otherwise omitted |

### 2.9.3 Generation on Import

BibTeX generation is the final step of every import path: arXiv ID import, DOI import, PDF file import, and Semantic Scholar search import. It is a pure in-memory string concatenation—no LLM calls, no network access, no external service dependency; the time cost is negligible. Legacy papers with a NULL BibTeX field can be backfilled in bulk via the "Backfill BibTeX" button on the Settings page.

> **Tip** BibTeX key uniqueness is enforced by the database. If you use Zotero or another tool that generates keys in a different format, LitFolio's keys will not collide—because LitFolio keys always start with a lowercase surname and never contain underscores. Generation is pure in-memory string concatenation with negligible overhead.

## 2.10 Citation Formatting

In addition to BibTeX, LitFolio can produce formatted citations in standard academic styles. In the paper drawer, click "Copy Citation" and choose from four styles. Formatting is pure template rendering—metadata fields are substituted into predefined string templates; no LLM is involved.

### 2.10.1 Four Style Templates

#### APA 7th

Template: `<Surname>, <First Initial>. (<Year>). <Title>. <Journal>.`

Example output:

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS*.

Rules: for more than 20 authors, list the first 19 followed by ... and the last; title in sentence case; journal name in italics.

#### IEEE

Template: `[N] <First Initial>. <Surname> et al., ``<Title>,'' <Journal>, <Year>.`

Example output:

[1] A. Vaswani et al., "Attention is all you need," *NeurIPS*, 2017.

Rules: IEEE uses bracketed numbering; 3+ authors use `et al.`; title in Title Case; journal in italics.

#### GB/T 7714-2015

Template: `[N] <SURNAME> <First Initial>, <SURNAME> <First Initial>, ... <Title>[J]. <Journal>, <Year>.`

Example output:

[1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention is all you need[J]. NeurIPS, 2017.

Rules: Chinese national standard—surname in uppercase before initials; 3+ authors use `et al.`; document type identifier `[J]` for journals, `[C]` for conferences, `[D]` for dissertations. LitFolio selects `[J]` or `[C]` automatically based on whether the venue string contains `Conference/Proceedings/Workshop`.

#### Chicago 17th

Template: `<Surname>, <First>. <Year>. ``<Title>.'' <Journal>.`

Example output:

Vaswani, Ashish, Noam Shazeer, Niki Parmar, et al. 2017. "Attention Is All You Need." *NeurIPS*.

Rules: authors use full names (not abbreviations); 7+ authors list the first 7 followed by `et al.`

### 2.10.2 Batch Citation Formatting

Select multiple papers and click "Export Citations → Formatted Text". LitFolio generates a numbered reference list in the chosen style. Numbers are assigned in the order papers appear in the list (IEEE and GB/T 7714 require numbering; APA and Chicago sort by author-year).

## 2.11 Batch Citation Export

In the library, select papers in checkbox mode; a toolbar "Export Citations" button appears. Three formats are supported:

### 2.11.1 BibTeX (.bib)

Merges the BibTeX entries of all selected papers into a single `.bib` file. Each record is taken directly from `papers.bibtex` (generated by the algorithm in Section 2.9) and separated by blank lines. If a paper's BibTeX field is NULL (very old data), the entry is skipped and listed in the export report. The output file can be used directly with `bibtex` / `biber` / `biblatex`.

### 2.11.2 RIS (.ris)

RIS (Research Information Systems) is a universal reference exchange format. Each record consists of tagged lines:

```
TY  - JOUR
TI  - Attention Is All You Need
AU  - Vaswani, Ashish
AU  - Shazeer, Noam
PY  - 2017
```

```
JO  - NeurIPS
DO  - 10.48550/arXiv.1706.03762
ER  -
```

Tag mapping rules: `TY` is determined by venue (journal → `JOUR`, conference → `CONF`, preprint → `UNPB`); `AU` one author per line in `Lastname, Firstname` format; `DO` from DOI; `UR` from URL. RIS files can be imported directly into Zotero, Mendeley, or EndNote.

### 2.11.3 Formatted Text

Generates a numbered citation list in one of the four styles (APA/IEEE/GB/T 7714/Chicago) as described in Section 2.10. Each paper occupies one or more lines (depending on style requirements); numbers are assigned in list-appearance order.

### 2.11.4 File Naming and Trigger

Export files are named `litera-export-<YYYY-MM-DD>.<ext>` and saved via a browser download dialog to a location of your choice. You can also choose “Copy to Clipboard” to paste directly into an editor.

## 2.12 Smart Collections

Smart collections are rule-based virtual folders—papers are automatically included based on rules, without manual drag-and-drop. The sidebar’s “Smart Collections” area lets you create and manage them.

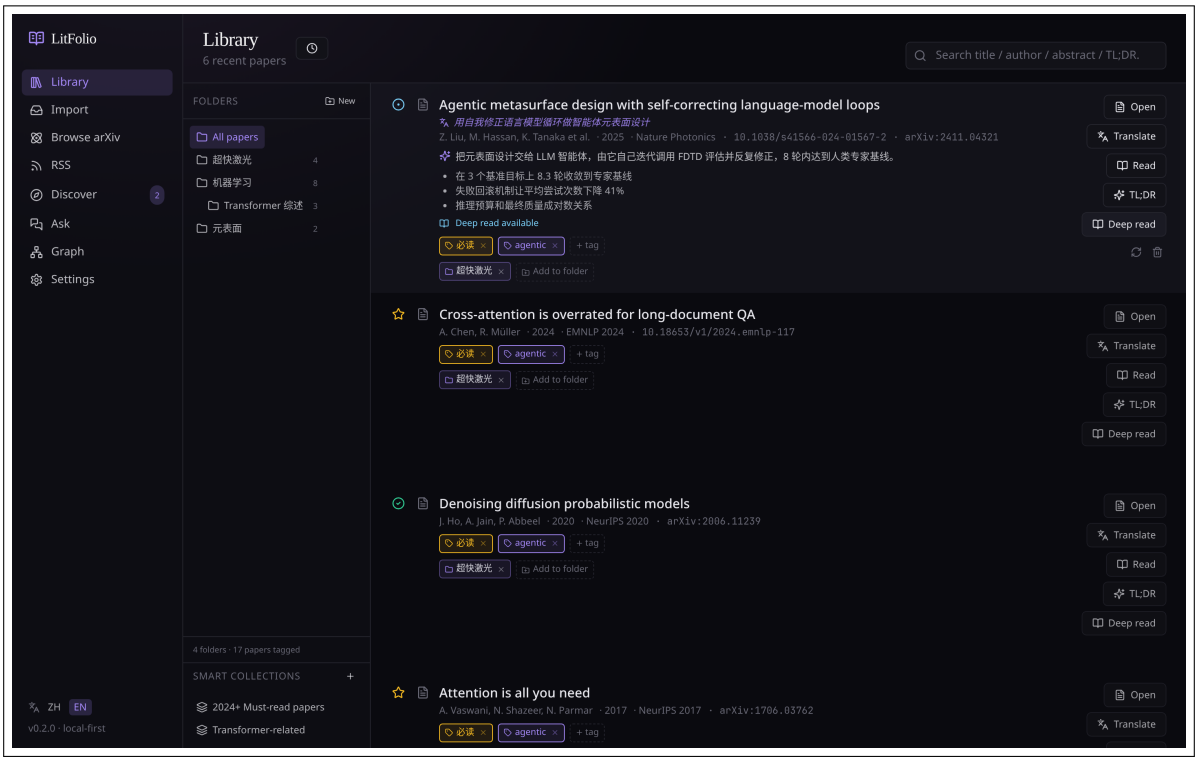


**Figure 2.4** Smart collection creation dialog. Papers are filtered by combining conditions; results update in real time.

### 2.12.1 Rule Structure

Each smart collection stores its rules as a JSON tree supporting nested AND/OR logic:

```
{
  "op": "and",
  "conditions": [
    { "field": "read_status", "op": "eq", "value": "unread" },
    { "field": "year", "op": "gte", "value": 2024 },
    { "field": "tags", "op": "includes", "value": "transformer" },
    { "op": "or", "conditions": [
      { "field": "folder", "op": "in", "value": "ML" },
      { "field": "title", "op": "contains", "value": "attention" }
    ]}
  ]
}
```

### 2.12.2 SQL Translation Engine

At query time, the rule tree is recursively translated into a SQL WHERE clause:

1. **Leaf nodes**—each condition becomes a SQL fragment:
   - `read_status` → `p.read_status = '<value>'`
   - `year` + `gte/lte` → `p.year ⩾ <value>` or `p.year ⩽ <value>`
   - `tags` + `includes` → `EXISTS (SELECT 1 FROM paper_tags pt JOIN tags t ON pt.tag_id = t.id WHERE pt.paper_id = p.id AND t.name = '<value>')`
   - `folder` + `in` → `EXISTS (SELECT 1 FROM paper_folders pf JOIN folders f ON pf.folder_id = f.id WHERE pf.paper_id = p.id AND f.name LIKE '<value>%')` (prefix matching to include subfolders)
   - `title` + `contains` → `p.title LIKE '%<value>%'`
   - `authors` + `contains` → `p.authors LIKE '%<value>%'`
2. **Branch nodes**—`op: "and"` becomes `(cond1 AND cond2 AND …)`, `op: "or"` becomes `(cond1 OR cond2 OR …)`. Nesting depth is unlimited.
3. **Final query**—assembled as `SELECT p.* FROM papers p WHERE <rules> ORDER BY p.added_at DESC`.

### 2.12.3 Real-Time Updates

Smart collections are not snapshots—they query live every time you select them. So when a paper's tags or status change, the next time you click the collection the results immediately reflect the update. This is fundamentally different from ordinary folders: folders require manual drag-in/drag-out; smart collections only care whether the rules match.

### 2.12.4 Performance

The translated SQL leverages SQLite indexes. For tag and folder conditions, the `paper_tags` and `paper_folders` tables have `paper_id` indexes, so EXISTS subqueries are fast. Ten smart

collections add no perceptible delay to sidebar loading—only the currently selected one executes its query; the rest display only their name and rule description.

> **Tip** Smart collections are ideal for long-lived perspectives such as “unread ML papers from the last two years” or “all must-read papers.” For ad-hoc filtering, the search box is more convenient. Rules can be edited at any time; matching results update immediately after modification.

## 2.13 Reading Queue

The reading queue is a prioritization tool independent of folders and tags, solving the problem of “I have 20 papers I want to read and need to order them.” Access it from the library sidebar under “Reading Queue.”

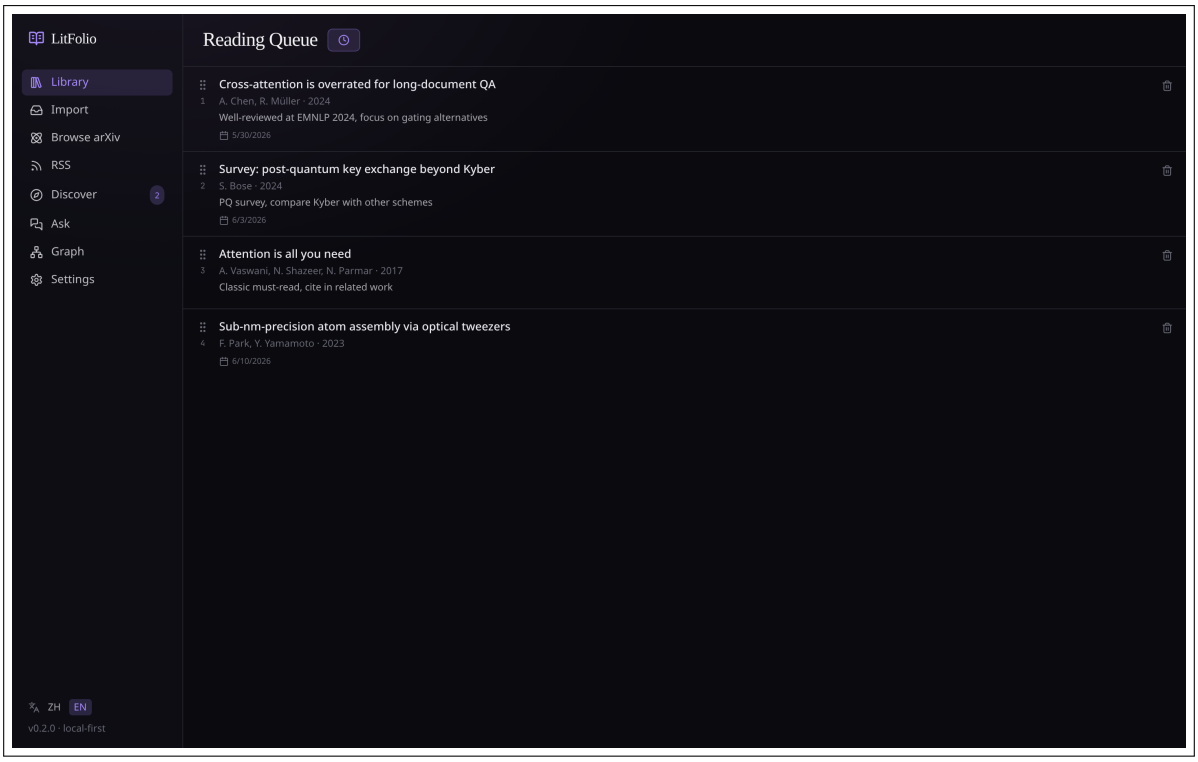


**Figure 2.5** Reading queue. Drag to reorder, set target dates, overdue papers are highlighted.

### 2.13.1 Queue Data Model

The queue is stored in the `reading_queue` table. Each row contains:

- `paper_id`—the associated paper (primary key; a paper can appear in the queue only once).
- `priority`—integer priority determined by drag-sort position (starting at 0, incrementing).
- `target_date`—optional Unix timestamp representing the planned reading date.
- `note`—optional note text.
- `added_at`—timestamp when the paper was enqueued.

### 2.13.2 Drag-and-Drop Sorting

The frontend uses the HTML5 Drag and Drop API for sorting. When a drag completes:

1. The new position index of the dragged item is computed.
2. All queue items have their `priority` values reassigned (starting at 0, incrementing by 1).
3. The `queue_reorder` command batch-updates the database.

Sort order persists across sessions—close the app and reopen it, and the order is unchanged.

### 2.13.3 Target Dates and Overdue Alerts

Target dates are optional. If the current date exceeds a paper's target date, that row is highlighted in amber (the same color used by `warnbox`). Overdue status is a visual cue only; it does not change any paper state automatically.

### 2.13.4 Automatic Status Update

When you open a queued paper in the reader, the system automatically transitions that paper's `read_status` from `unread` to `reading`. This transition is idempotent—if the paper is already `reading` or higher, nothing happens.

### 2.13.5 Queue vs. Folders/Tags

The queue is a completely independent dimension:

- A paper can be in any folder, carry any tag, and be in any reading state while also being in the queue.
- Removing a paper from the queue does not delete the paper itself or change its folders, tags, or status.
- If a queued paper is deleted, its queue entry is cascade-deleted.

## 2.14 Paper Deduplication

LitFolio automatically detects duplicate papers at import time, preventing the same paper from appearing multiple times due to different import sources (arXiv ID / DOI / PDF file).

### 2.14.1 Three-Level Matching Strategy

Detection runs in priority order and stops on first match:

1. **DOI exact match**—query `SELECT id FROM papers WHERE doi = ?`. DOIs are globally unique identifiers with a 100% match rate.
2. **arXiv ID exact match**—query `SELECT id FROM papers WHERE arxiv_id = ?`. Different versions of the same paper (`v1`/`v2`) have their version suffix stripped before comparison.

3. **Title similarity match**—uses normalized Levenshtein edit distance. Given two titles $s_1, s_2$, the edit distance $d(s_1, s_2)$ (each insertion, deletion, or substitution costs 1) is normalized as:

$$D(s_1, s_2) = \frac{d(s_1, s_2)}{\max(|s_1|, |s_2|)}$$

When $D < 0.15$, the papers are considered duplicates. This means two titles can differ by up to 15% of their characters—enough to tolerate case differences (`Attention Is All You Need` vs. `attention is all you need`), punctuation differences (hyphens vs. spaces), and minor OCR errors.

### 2.14.2 Title Preprocessing

Before computing edit distance, both titles undergo preprocessing:

1. Convert to lowercase.
2. Remove all non-alphanumeric characters (keeping spaces).
3. Collapse consecutive spaces into a single space.
4. Trim leading and trailing whitespace.

This ensures that `"Attention-Is-All-You-Need"` and `"attention is all you need"` have a distance of 0 (exact match).

### 2.14.3 Full-Library Scan

The "Find Duplicates" button on the Settings page performs a full-library scan. The algorithm is $O(n^2)$ title comparisons (where $n$ is the number of papers), with two optimizations:

- **DOI/arXiv ID pre-filter**—papers with DOIs are bucketed by DOI first; title comparison only happens within the same bucket. The same applies to arXiv IDs. This dramatically reduces the number of pairs needing string comparison.
- **Length pre-filter**—if the length difference between two titles exceeds 15%, the edit-distance computation is skipped entirely. This is a conservative pruning: when $|s_1| - |s_2| > 0.15 \times \max(|s_1|, |s_2|)$, $D$ is necessarily $> 0.15$.

A full scan of 1,000 papers takes about 2–3 seconds on typical hardware.

### 2.14.4 Merge Workflow

When duplicates are found, a merge dialog appears. You choose which record to keep (usually the one with more complete metadata), and the system performs:

1. Transfer all highlights, notes, terms, tags, and folder memberships from the merged paper to the retained paper.
2. Set the retained paper's reading state to the higher of the two (`must` > `read` > `reading` > `unread`).
3. Fill empty TL;DR, deep-read, and translation fields on the retained paper with values from the merged paper (existing values are not overwritten).
4. Delete the merged paper's database row and `papers/<id>/` directory.

> **Warning** Merge operations cannot be undone. The merged paper's PDF file is moved to `backups/deleted/`, but metadata is deleted directly. If you are unsure which record to keep, cancel the merge, manually inspect both records, and try again.

Chapter 3

# Importing Papers

The Import page (`/import`) consolidates three entry points: arXiv/DOI metadata, local PDFs, and Semantic Scholar search. All three ultimately write to a `papers/<id>/` directory and a SQLite row.

## 3.1 arXiv ID / DOI

The most common import path. It is a **two-step** process:

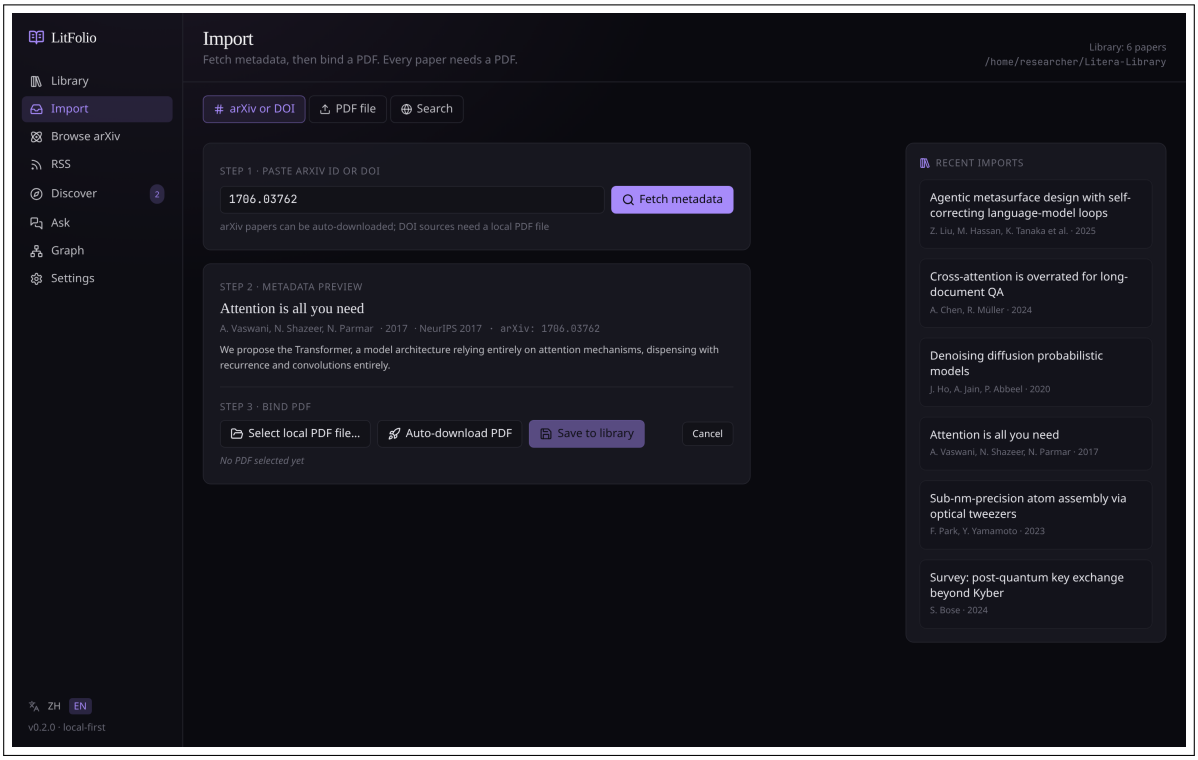


**Figure 3.1** arXiv/DOI tab. Paste an ID or DOI, click "Query Metadata" to retrieve title/authors/abstract, then click "Import" to persist.

1. **Query metadata**—paste an arXiv ID like `1706.03762` or a DOI like `10.1038/s41586-024-...` and click Query. LitFolio fetches metadata from the arXiv API or Crossref and immediately displays the title, authors, and abstract.
2. **Import**—confirm the metadata and click "Import." LitFolio will simultaneously attempt to download the PDF (directly from arXiv for arXiv IDs; via Unpaywall's open-access pathway for DOIs—a download failure does not block import).

> **Tip** arXiv IDs support both the new format (`2402.12345`) and the old format (`cs/0501001`). Versioned IDs like `2402.12345v2` are also parsed correctly.

## 3.2 Local PDF Files

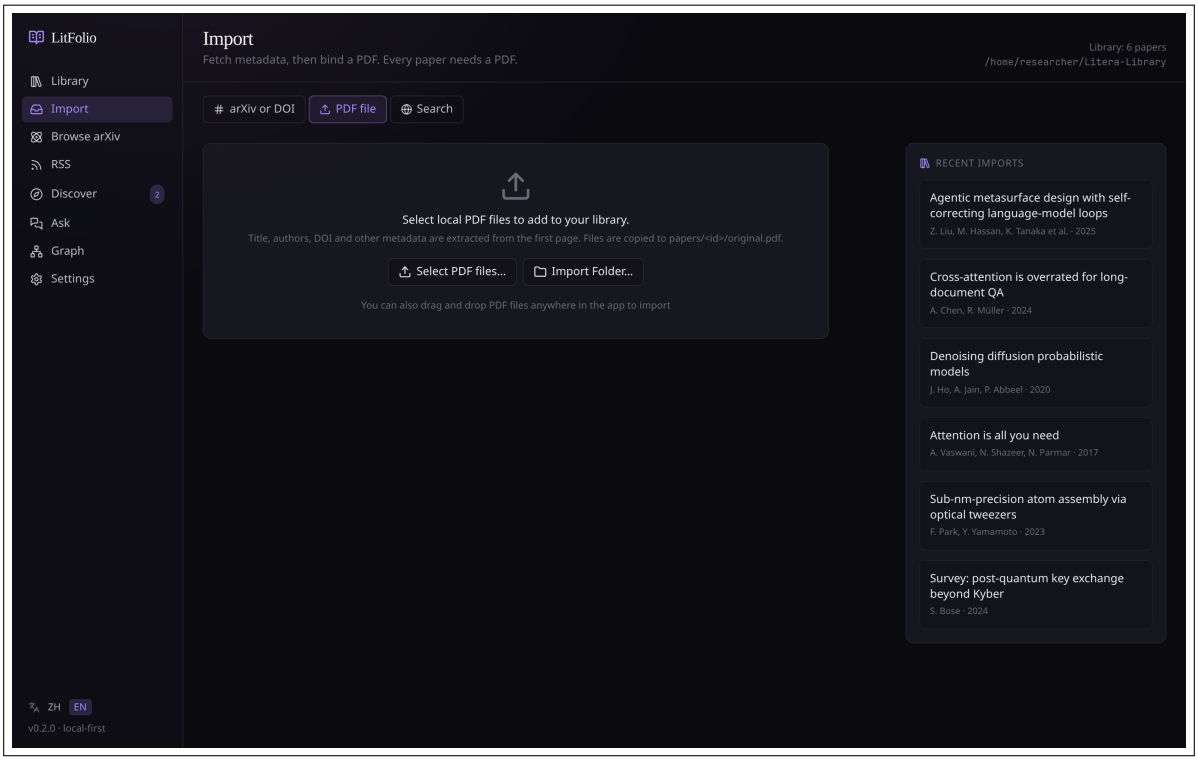


**Figure 3.2** PDF file tab. Select a single PDF or an entire directory; edit the guessed title/author before importing.

- **Single PDF**—after selecting a file, LitFolio runs lightweight metadata extraction (first page of the PDF + DOI regex) to guess title and authors; you can edit them in the form.
- **Drag and drop**—drag a PDF onto any LitFolio page to jump straight into the import flow.

### 3.2.1 PDF Metadata Extraction Strategy

When importing a local PDF, LitFolio tries the following extraction steps in order:

1. **PDF Info Dictionary**—use `lopdf` to read the `/Title`, `/Author`, and `/Subject` fields. Text decoding attempts both UTF-8 and UTF-16 BE (with BOM), since academic PDFs commonly use either encoding.
2. **First-page DOI regex**—if the Info Dictionary contains no DOI, scan the first-page text with the regex `10.\d{4,9}/\S+`, stripping trailing punctuation (.,);] etc.) from matches.
3. **Title guess**—if still no title, take the first line on page 1 that is $\geq 12$ characters long and contains letters.
4. **Filename fallback**—if everything else fails, use the PDF filename (without extension) as the title.

The author field is split on commas, semicolons, and “and” (accommodating both BibTeX and natural-language formats). A SHA-256 hash of the file is also computed at import time for deduplication.

## 3.3 Semantic Scholar Search

If you only remember a fragment of the title, a keyword, or want to find all papers by a specific author:

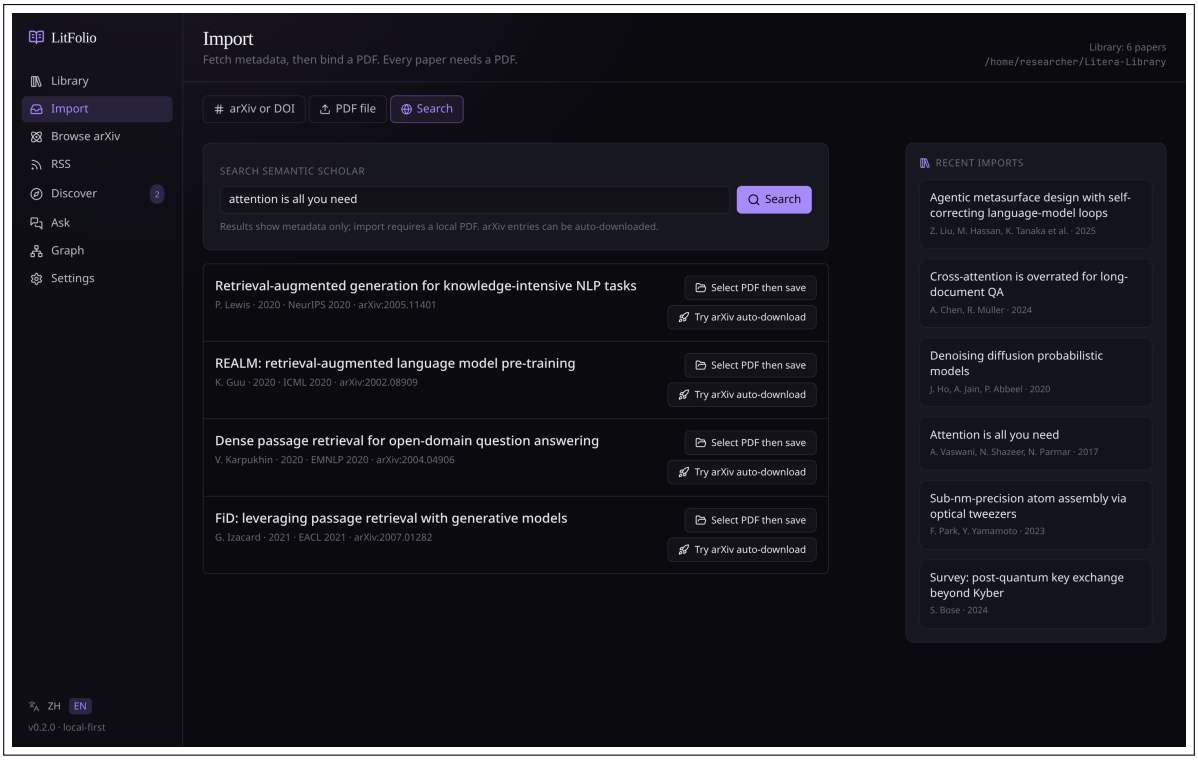


**Figure 3.3** Search tab. Results are sorted by citation count; each can be directly “Imported” (fetches metadata + attempts PDF download).

LitFolio calls the Semantic Scholar Graph API, returning 25 results per page. After selecting a result and clicking “Import,” the full metadata is fetched and the PDF download is attempted.

> **Warning** The Semantic Scholar public API has QPS limits; frequent searching may trigger 429 responses. If your research direction requires heavy retrieval, consider applying for a free API key on their website and entering it under “Settings → API Keys.”

## 3.4 Batch Folder Import

Beyond single-PDF import, LitFolio supports importing all PDFs in a directory. On the Import page, click the “Import Folder” button and select a directory.

### 3.4.1 Recursive Scanning

After selecting a folder, LitFolio recursively scans all subdirectories for `.pdf` files. The scan results are displayed as a list, each entry showing:

- Filename (used as a fallback source for title guessing).
- File size.
- Extraction status (Pending / Processing / Success / Failed).

### 3.4.2 Filename Heuristic Parsing

For batch-imported PDFs, LitFolio attempts to extract metadata from the filename. Common naming patterns:

**`Author_Year_Title.pdf`** Split on underscores: first segment is the author, second is the year (4 digits), the rest is the title.

**`Author - Title (Year).pdf`** Split on ` - ` to separate author and title; extract the year from parentheses.

**`2024 - Author - Title.pdf`** Year-first variant.

Heuristic parsing achieves roughly 70% accuracy. For files that cannot be parsed, the filename (minus extension) is used as the title, and the author and year are left blank for manual completion after import.

### 3.4.3 Concurrent Processing and Progress Tracking

Batch import uses 4-way concurrent processing (configurable). Progress is pushed to the frontend in real time via event streams:

- A progress bar shows "Processing 23/150... (3 failed)."
- Failed files are listed separately with failure reasons (corrupted PDF, no text layer, metadata extraction failure, etc.).
- On completion, a summary is displayed: N succeeded, N failed, N skipped (duplicate papers already in the library).

> **Tip** Batch-importing 100 PDFs typically completes within 5 minutes. If some PDFs are scanned images (no text layer), LitFolio will still import them (using the filename as the title), but full-text search and RAG retrieval will not cover their content. You can preprocess scanned PDFs with `ocrmypdf` and then re-bind them.

## 3.5 Redirect from RSS / arXiv Browse

The arXiv browse and RSS feed features described in Chapters 5 and 6 each include an "Import" button in their detail drawers, which is equivalent to pasting the arXiv ID on the Import page. In other words: you do not need to visit the Import page separately—when browsing RSS, just import directly from what catches your eye.

Chapter 4

# The Reader

The reader is LitFolio's second most-used page. Access it from the main list by clicking the `clip` icon or "Open in Reader."

## 4.1 Three-Pane Layout

### 4.1.1 PDF Text Extraction and Caching

When the reader loads a PDF, PDF.js performs full-text extraction on the frontend (it handles CMap encoding, embedded font subsets, and ToUnicode tables in modern academic PDFs more reliably than the backend `lopdf`). The extracted text is sent to the backend via IPC and cached in `papers/<id>/text.txt`. Subsequent term extraction and FTS5 indexing read from this cache file directly—as long as `text.txt`'s modification time is newer than `paper.pdf`'s, the cache is reused; otherwise `lopdf` re-extracts. This means term generation never needs to re-parse the PDF and is available instantly.

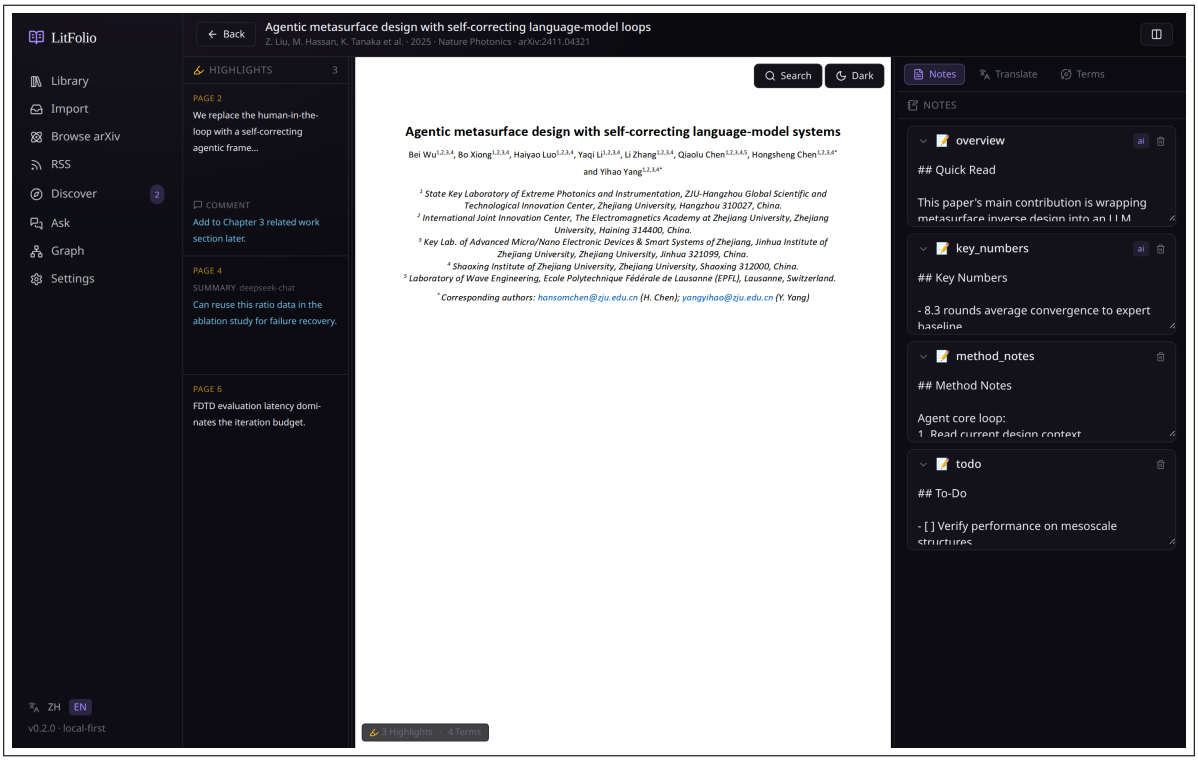


**Figure 4.1** Reader three-pane layout. Left: highlight list; center: PDF.js-rendered document; right: notes / selection translation / terms tab switcher. Pane widths are adjustable by dragging; state is automatically persisted.

- **Left pane**—all highlights for the current paper (sorted by page); click any highlight to jump to its location.

- **Center pane**—the PDF body. Based on PDF.js + react-pdf-highlighter; inherits system dark mode and inverts colors during rendering.
- **Right pane**—three online tool tabs: **Notes** (Markdown), **Selection Translation**, and **Terms**.

## 4.2 Highlights: Text Selection + Region

Two highlight methods:

**Text selection highlight** Select text in the PDF as you normally would; releasing the mouse shows a small popup—click "Highlight." The selected text is cleaned of ligatures and line breaks and written to the highlights table.

**Region highlight (`Alt+drag`)** Hold Alt and drag a rectangle over the PDF to capture a screenshot as a highlight—useful for formulas, charts, and diagrams.

Each highlight in the left pane supports comments (click the pencil icon); comments are written in Markdown.

## 4.3 Selection Translation

Select text in the PDF, switch to the "Selection Translation" tab in the right pane, and click "Translate Selection":

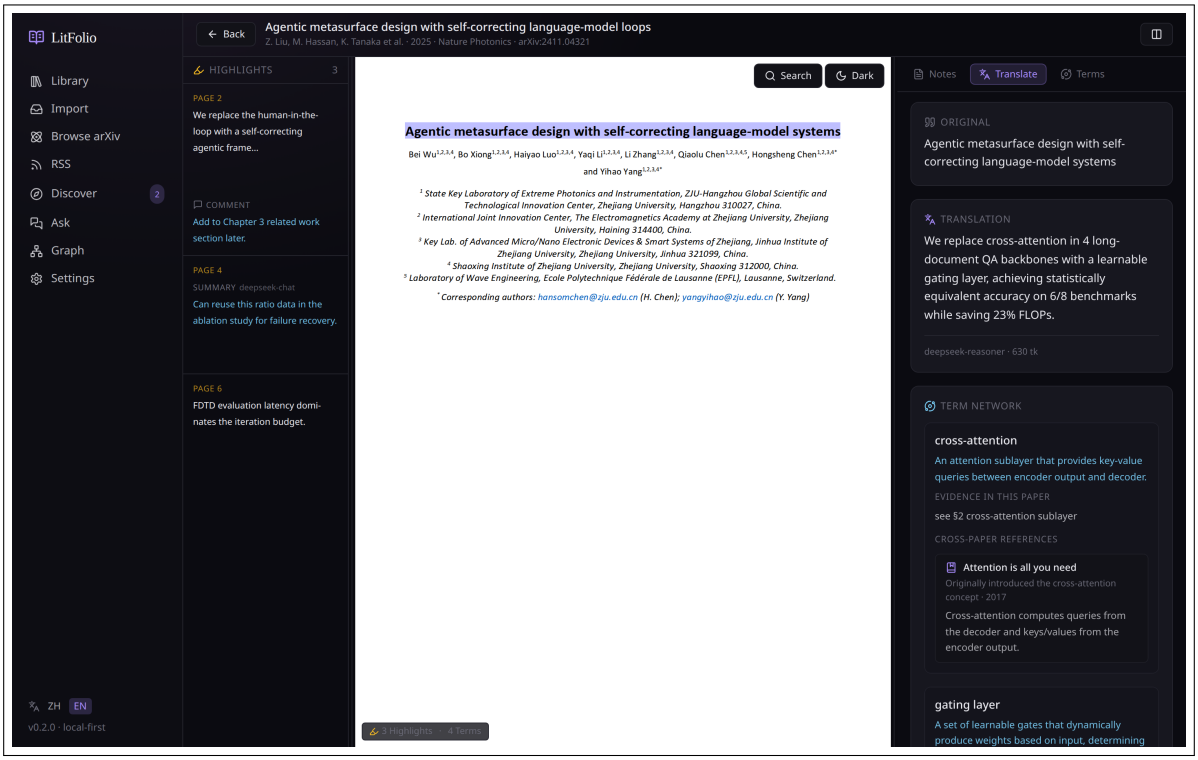


**Figure 4.2** Selection translation. In addition to the bilingual side-by-side view, domain terms from the passage are listed as purple chips—click any chip to jump to the "Terms" tab for the full definition.

The output is structured as three parts: bilingual text, term extraction, and cross-paper references.

#### Internal Pipeline

Selection translation is not a single API call but a three-step process:

1. **Term matching**—the selected text is checked against the current paper's existing terms (case-insensitive substring matching on normalized forms). Matches reuse existing definitions and evidence snippets; misses trigger real-time extraction: three regex patterns capture all-uppercase abbreviations (e.g. SSIM), Title Case proper nouns (e.g. MetaDesigner), and 1–3 word noun phrases, filtered through `is_term_candidate`.
2. **Cross-paper association**—for each term, an FTS5 search across the entire library (max 12 results) is performed. After excluding the current paper, up to 3 related papers are retained. Association type is determined by which section of the target paper the term appears in: method section → "Method Implementation," comparison section → "Comparative Discussion," other → "Related Mention."
3. **LLM translation**—the selected text (truncated to 2000 characters) is sent to the translation model. The prompt injects a term glossary (matched terms with their local definitions) and instructs the LLM to keep English terms verbatim, annotating them on first appearance as `Term[Chinese gloss]`. This keeps the translation fluent while preserving term anchors.

The frontend composites the three results: bilingual text + purple term-chip list + cross-paper links.

- **Bilingual text**—original and translated paragraphs rendered side-by-side.
- **Terms**—domain terms extracted from the passage, each with a one-line in-context definition; if other papers also use the same term, a "cross-paper reuse `N`" badge is shown.
- **Cross-paper references**—locations in the current paper where the same concept was previously discussed (page + line); useful for the "Have I seen this before?" scenario in long papers.

## 4.4 Term Table and Term Network

"Generate Terms" runs the LLM `term` task (bound by default to your configured term model). Generated terms are cached in the `papers/<id>/` directory and the SQLite `terms` table for reuse during selection translation.

### 4.4.1 Term Extraction Algorithm

Term generation does not simply dump the full text into an LLM and ask it to list terms. LitFolio uses a **local statistical filtering + LLM definition** two-stage architecture: first, an offline TF-IDF statistical method extracts candidate terms from the full text; then the LLM writes a one-line definition for each candidate. The advantages: it saves tokens (no need for the LLM to scan the full text) and the candidate list is reusable for term matching during selection translation.

#### Candidate Extraction: Weighted TF x IDF x Multi-Factor Scoring

The candidate pipeline has four steps:

1. **Weighted segmentation**—different sections of the paper are assigned different weights (title 3.0, abstract 2.0, methods 2.0, research questions 1.5, datasets 1.5, comparison

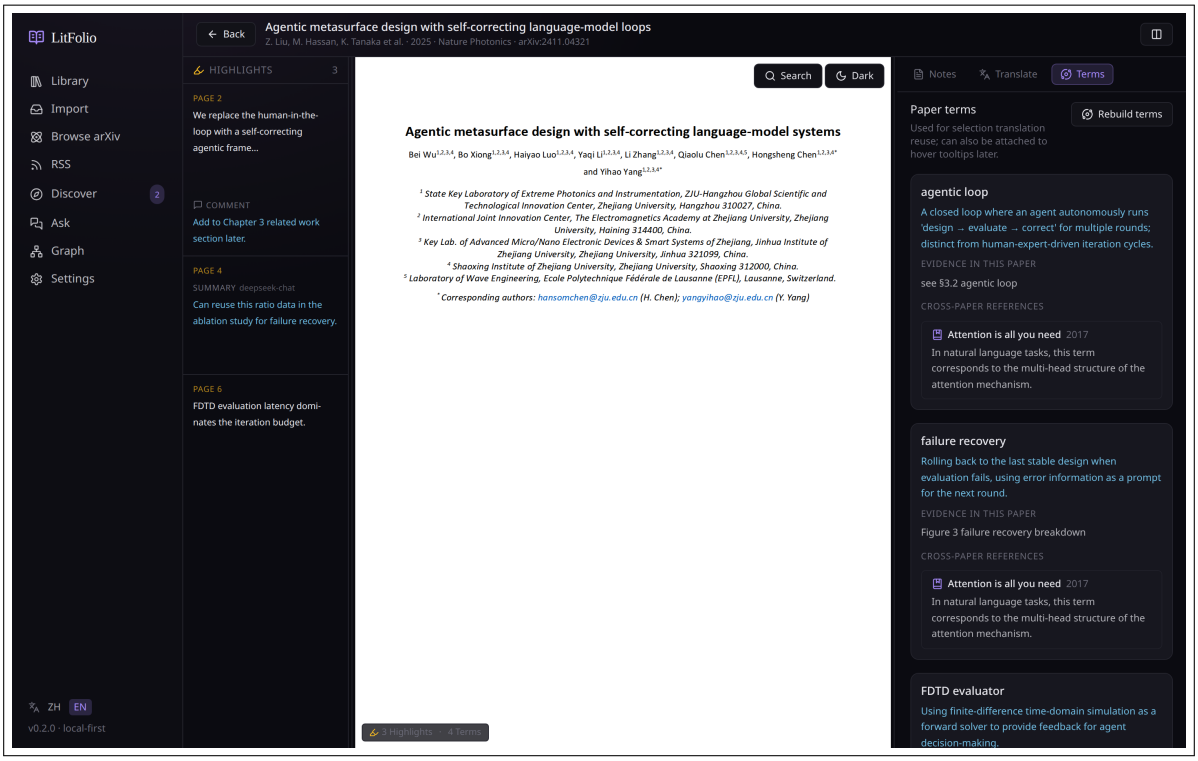


**Figure 4.3** Terms tab. After generating the paper's term table, each term shows "In-paper evidence / Cross-paper reuse" sections, facilitating cross-paper consistency checks.

1.0, limitations 1.0, TL;DR 1.0, full PDF text 0.4). Terms appearing in the title or abstract carry higher weight; those in the body carry lower weight.

2. **Regex extraction + filtering**—regex matching is applied to each segment: by default only single tokens are captured (proper nouns, all-uppercase abbreviations, hyphenated compounds); for "Full Name (ACRO)" abbreviation pairs, a dedicated recognition path allows the full name up to 5 words. Each candidate must pass `is_term_candidate`: the first and last words cannot be function words (with/from/of...), 3+ word phrases cannot have conjunctions/prepositions in the middle (and/or/with...), and all-lowercase single words must be at least 4 characters and contain an uppercase letter or hyphen.

3. **TF-IDF calculation**—term frequency is the sum of candidate occurrences in each weighted segment multiplied by the segment weight; document frequency is the number of papers among the library's most recent 500 that contain the candidate. The IDF formula is $\ln\big((N+1)/(df+1)\big)+1$. Three correction factors are applied:

   - **section_hits** ($\sqrt{\min(3, \text{number of sections appeared in})}$) —appearing in more distinct sections increases credibility.
   - **surface_quality**—all-uppercase abbreviations $\times 2.0$, Title Case multi-word $\times 1.6$, hyphenated $\times 1.3$, all-lowercase multi-word $\times 0.7$. Surface forms that look more like technical terms score higher.
   - **abbrev_bonus** ($\times 1.8$)—candidates from the abbreviation-pair path receive extra weighting.

4. **Sort and truncate**—candidates with a total score $\geq 0.9$ are sorted in descending order and the top 12 are kept. Scores below the threshold are discarded (for example, a single-occurrence all-lowercase bigram scores about 0.7, just below the cutoff).

#### Abbreviation Pair Recognition

For text like "Structural Similarity Index Measure (SSIM)," LitFolio uses a dedicated regex to capture the full name and abbreviation, then performs two checks:

- **Initial matching**—take the first letter of each word in the full name and greedily match against the abbreviation character by character; at least half must match. Hyphens are treated as word boundaries ("Retrieval-Augmented Generation" → RAG matches).
- **Full-name refinement**—if the regex-captured full name is longer than the abbreviation, a word sequence from the tail equal to the abbreviation length is extracted for exact initial matching; if that misses, stop words (and/the/of) are removed and the process retries. For example, "average structural similarity index measure (SSIM)" is refined to "structural similarity index measure."

Only the abbreviation enters the final term list (the full name does not appear independently), but the full name is saved as context for the LLM definition.

#### Author Name Filtering

A frequently overlooked noise source is author surnames—common surnames like "Chen," "Yang," and "Li" appear at high frequency in paper author lists, satisfy Title Case proper-noun criteria, and can rank highly among term candidates. Before extraction, LitFolio collects all author surnames of the current paper into a HashSet (splitting on spaces, hyphens, and periods, then lowercasing). Candidates that match after normalization are skipped. This is especially important for Chinese authors' pinyin surnames—"Chen" and "Wang" appear at extremely high frequency in author lists and would dominate the term rankings without filtering.

#### LLM Definition Generation

Once candidates are finalized, each term's "surface form + full name (if any) + evidence snippet" is assembled into a prompt for the LLM, which returns a JSON-formatted one-line Chinese definition. If the LLM call fails or returns empty, a fallback is used: `This paper uses <term> in the context of the current argument or method.`

> **Tip** Term generation consumes significantly more tokens than TL;DR (it must scan the full text). Consider binding it to a cost-effective domestic low-tier conversational model, or running a local 7B+ open-source model.

## 4.5 Structured Note Cards

The "Notes" tab has been upgraded from a single text box to a **structured card system**. Each paper has five independently editable note sections by default, stored in the `paper_note_sections` table.

### 4.5.1 Default Sections

**Problem** What problem the paper addresses.

**Method** What method is proposed.

**Key Numbers** Key experimental data or metrics.

**Limitations** Limitations and open issues.

**Thoughts** Your own ideas and comments.

Each section is independently editable, collapsible, and reorderable by drag-and-drop. Sort order is persisted through the `sort_order` field. You can add custom sections (e.g., "Experiment Ideas," "Related Work"); custom sections have the same editing and sorting capabilities as the defaults.

### 4.5.2 AI Auto-Fill Mechanism

Each section has a `source` field indicating the content origin:

**user** Content you wrote manually.

**ai:tldr** Auto-filled by TL;DR (Quick Read).

**ai:quick_read** Auto-filled by Deep Read.

The fill rule is **fill only empty, never overwrite**:

1. When TL;DR runs, the `key_findings` array returned by the LLM is written to the "Key Numbers" section—but only if that section's `content` is empty and its `source` is not `user`.
2. When Deep Read runs, the four-section JSON is mapped to the corresponding sections: `problem` → Problem, `method` → Method, `limitations` → Limitations. Only empty sections are filled.
3. If you have manually edited a section (`source = user`), AI will never overwrite it, even if you re-run TL;DR or Deep Read.
4. AI-filled content is prefixed with a timestamp: `[2025-05-27 AI:TLDR] <content>`.

### 4.5.3 Sections and Full-Text Search

All sections' `content` fields are included in the `papers_fts` FTS5 index. This means you can search for a keyword you wrote in a section and find the corresponding paper—even if the word never appears in the title or abstract.

> **Tip** Section drag-and-drop sorting uses HTML5 Drag and Drop; sort order is written to the `sort_order` field. The default order is Problem → Method → Key Numbers → Limitations → Thoughts.

## 4.6 Annotation Color Semantics

Highlight colors can carry semantic meaning, stored in the `highlights.label` field. Configure color-to-label mappings under "Settings → Annotation Colors." Default mapping:

| Color | Label | Semantics |
|---|---|---|
| Yellow | Key Finding | Key findings, important conclusions |
| Purple | Method | Methodological points, algorithmic details |

| Color | Label | Semantics |
|---|---|---|
| Blue | To Verify | Claims that need verification, questionable data |
| Red | Disagree | Points of disagreement, controversial assumptions |
| Green | Citable | Excellent formulations worth quoting directly |

### 4.6.1 Workflow

1. **Create highlights**—after selecting text, the popup menu shows each color with its label name (e.g., "Yellow · Key Finding"). Choosing a color simultaneously sets the label.
2. **Filter highlights**—the top of the reader's left pane now has a row of label filter chips. Clicking a chip shows only highlights with that label; click again to remove the filter. Multiple chips can be active (OR semantics).
3. **AI integration**—the highlight summary feature (invoke LLM on selected highlights) supports label-based filtering. For example, summarize only "Key Finding" highlights, ignoring "To Verify" noise.
4. **Export integration**—during Markdown export, highlights are grouped by label, with each label as a third-level heading.

### 4.6.2 Custom Labels

You can modify the color-to-label mapping in Settings: rename labels, add new colors, or delete unused mappings. Existing highlights are unaffected—the label name is stored in each highlight record, not referenced from the global configuration.

## 4.7 Markdown Notes

The bottom of the "Notes" tab still has a free-text Markdown editing area, auto-saved to `papers/<id>/note.md`. LitFolio does not enforce any template—you can use any Markdown syntax and embed arbitrary image links; the file is synced along with the PDF via WebDAV.

## 4.8 PDF Search

`Ctrl+F` (macOS: `Cmd+F`) opens the search box:

- `Enter` jumps to the next match; `Shift+Enter` jumps to the previous.
- Match count is displayed to the right of the search box.
- `Esc` closes the search box.

## 4.9 Page Shortcuts

Other commonly used keys in the reader:

- `j` / `k`—next page / previous page.
- `1` / `2` / `3`—switch the right-pane tab.
- `[` / `]`—adjust left/right pane width (5% step).

The full shortcut table is in **Chapter 17**.

# Part II

# Discovery and Synthesis

Chapter 5

# arXiv Browse

The `/browse` route is a page designed for researchers who like to ”browse arXiv.” It organizes papers by the official arXiv category tree, letting you explore the latest papers in a field without having specific keywords in mind.

## 5.1 Category Tree

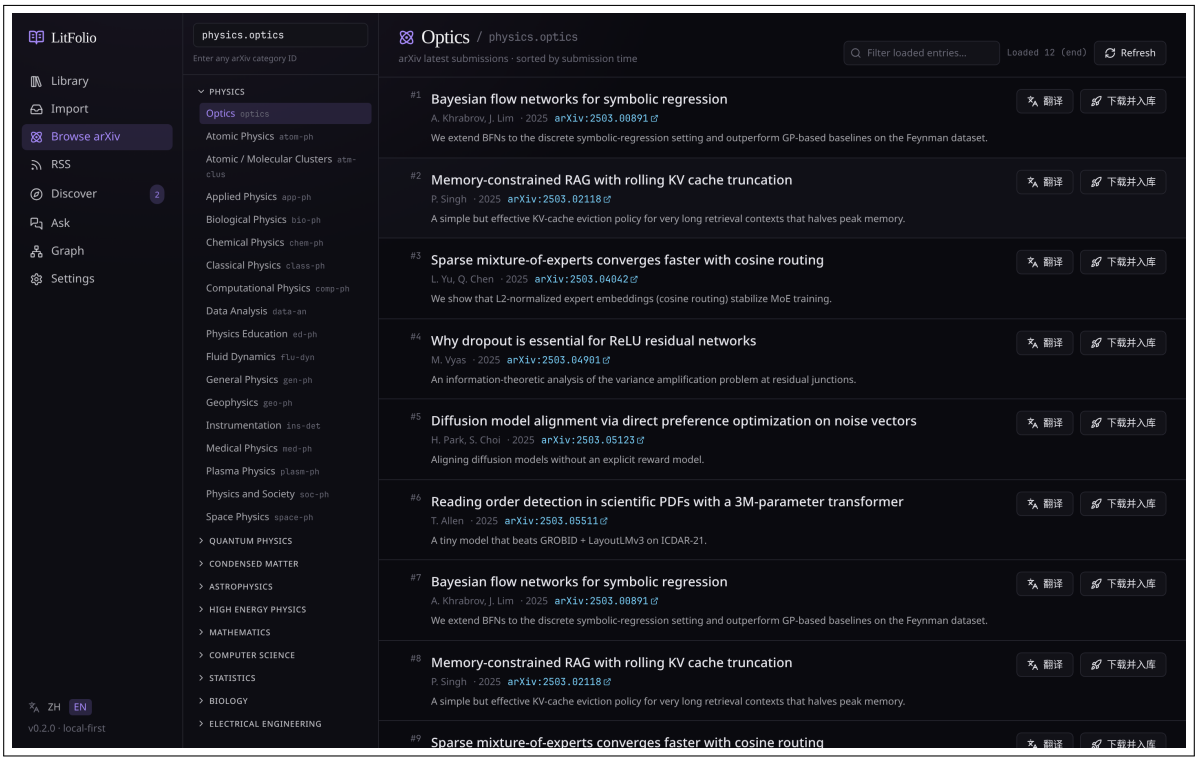


**Figure 5.1** arXiv Browse. The left sidebar shows the category tree (top-level groups such as cs / math / physics); the main area displays the latest papers in the selected category, with pagination and import support.

LitFolio includes a built-in Chinese–English mapping for all 150+ arXiv categories. Clicking any leaf node (e.g. `cs.LG` for Machine Learning, `physics.optics` for Optics) triggers an arXiv API request to fetch the 50 most recent papers.

## 5.2 Pagination and ”Exhausted”

Scrolling to the bottom and clicking ”Load More” fetches the next page (50 items per page). When a category has low publication volume and two consecutive requests return

no new entries, LitFolio displays ”Exhausted” and stops requesting—this is intentional behavior to avoid unnecessary API calls, not a bug.

## 5.3 Importing

Clicking any paper opens a detail drawer showing the full abstract, authors, submission date, and original arXiv link. The ”Import” button in the bottom-right corner of the drawer follows the same arXiv ID import workflow described in Chapter 3—the only difference is that the ID is already filled in.

> **Tip** The arXiv Browse page itself does **not** save the list to the local database—it queries the live API every time. If you want to follow a category's new papers long-term, consider using RSS subscriptions (see the next chapter), which work offline, support translation, and track read status.

Chapter 6

# RSS Feeds

The `/feeds` route lets you track arXiv sub-categories, journal articles, and research group homepages via RSS. All fetched entries are stored in SQLite, with unread counts and read status saved locally.

## 6.1 Feed Management

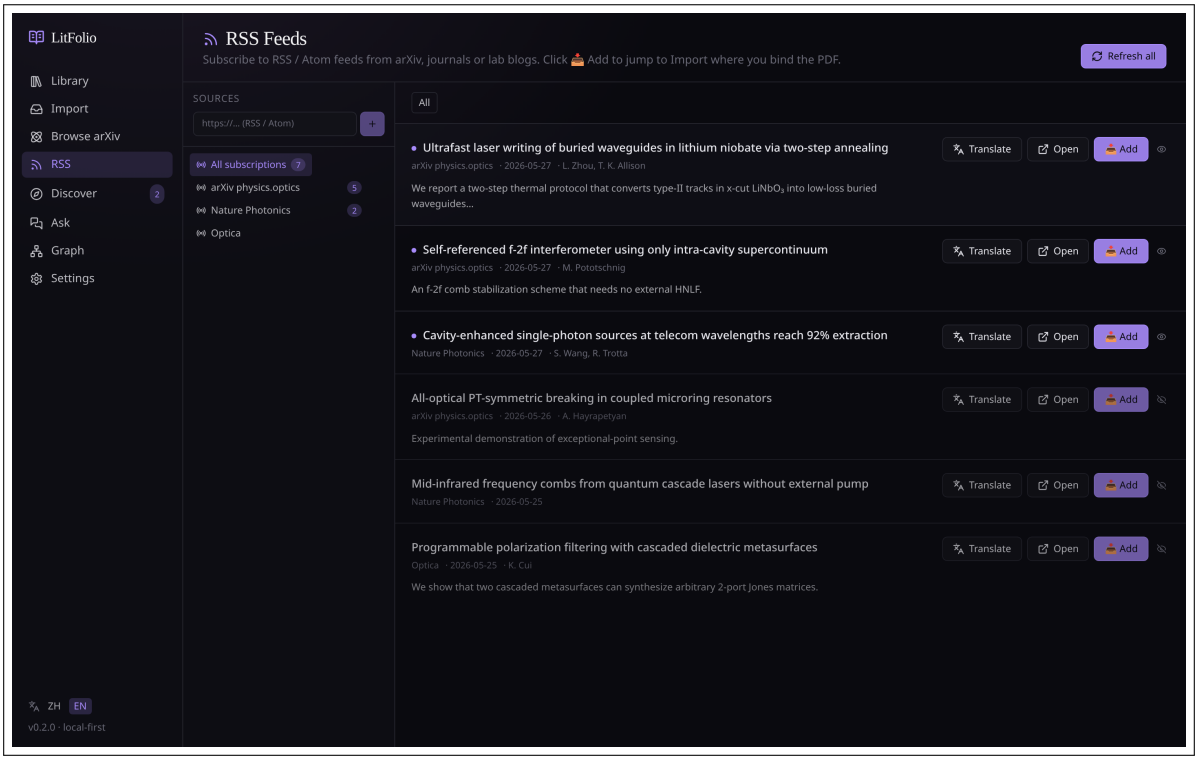


**Figure 6.1** Feed list. Left: sources (arXiv cs.LG / Nature Photonics / Optica, etc.); Right: latest entries for the selected source, with unread badges, translation buttons, and import buttons.

To add a new feed source, click the ”+” button and enter the RSS URL and a display name. LitFolio imposes no restrictions on feed sources—any valid RSS/Atom feed works. See Appendix C for commonly used sources.

## 6.2 Conditional GET Refresh

LitFolio uses conditional GET (`If-Modified-Sometimes` / `ETag`) to fetch feeds. When the remote has not been updated, the server returns 304 Not Modified—this is why clicking ”Refresh” sometimes instantly responds with ”Already up to date.” The practical implications are:

- Refreshing frequently is **cheap**—it will not trigger rate limiting on the server.
- Genuine ”nothing new” and ”network/server error” states display the same message; you need to check the error badge to tell them apart.

## 6.3 Entry Details and Translation

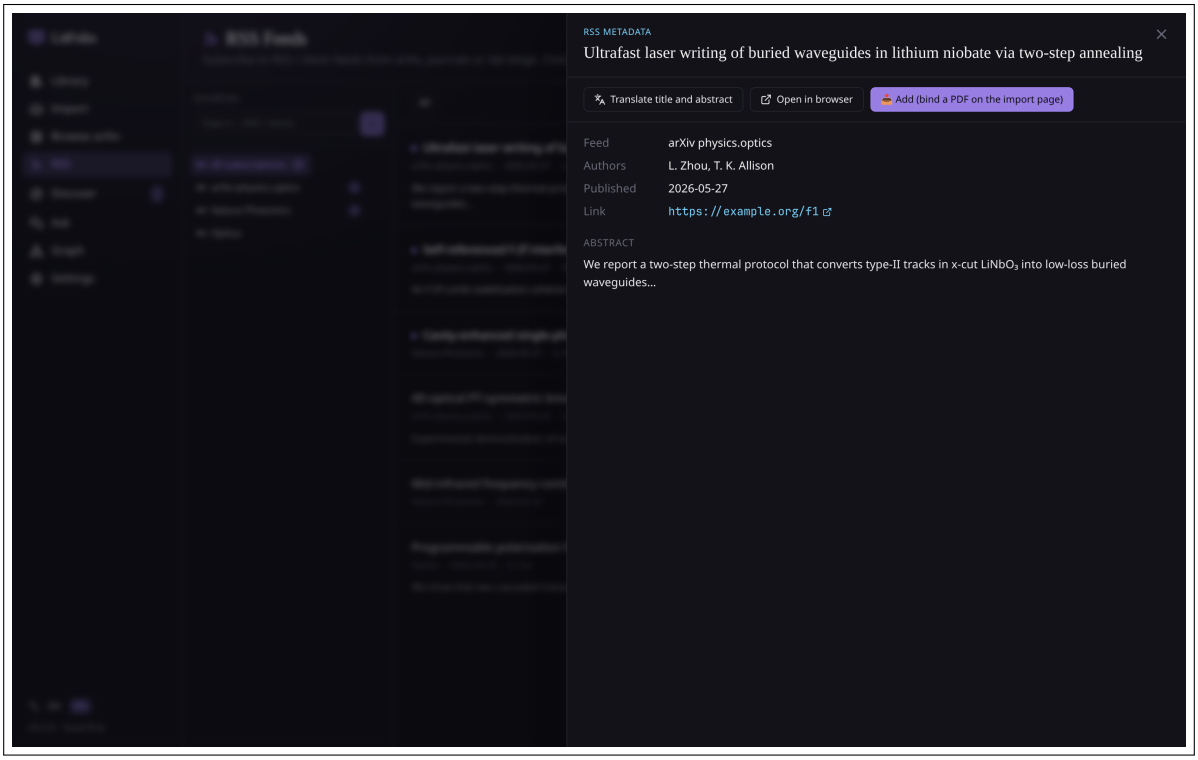


**Figure 6.2** After selecting a feed item, the right drawer shows the full abstract; you can translate with one click, mark as read, or import directly (equivalent to pasting an arXiv ID on the Import page).

The ”Translate” button works the same way as in the library: it invokes the `translate` task (bound to the translation model you configured). Results are cached in the SQLite `feed_items` table and load instantly on subsequent views.

> **Warning** RSS translation consumes tokens for every entry. If your subscribed sources update frequently (e.g., multiple arXiv categories), consider binding the translation task to a **local Ollama open-source chat model** or a **low-cost domestic API** to avoid runaway costs.

## 6.4 Direct Import from Feed Items

The ”Import” button in the detail drawer follows the exact same workflow as the arXiv ID import process in Chapter 3—fetching full metadata and attempting to download the PDF. After import, the entry remains in the feed list but is marked as ”Imported.”

Chapter 7

# Topic Discovery

The `/topic` route is one of LitFolio's advanced capabilities, split into two tabs: **Search Recall** and **Survey Generation**. They solve different problems.

## 7.1 Two-Tab Mental Model

**Search Recall** You already have a rough research direction and want to find all relevant papers from your existing library. Workflow: query expansion → multi-path FTS5 retrieval → rank by citation count. Fully offline, no external network calls.

**Survey Generation** You want a "bird's-eye view" of a new direction, but your library may not contain any papers on that topic. Workflow: LLM decomposes into sub-topics → Semantic Scholar fetches real papers → LLM annotates must-reads.

## 7.2 Search Recall

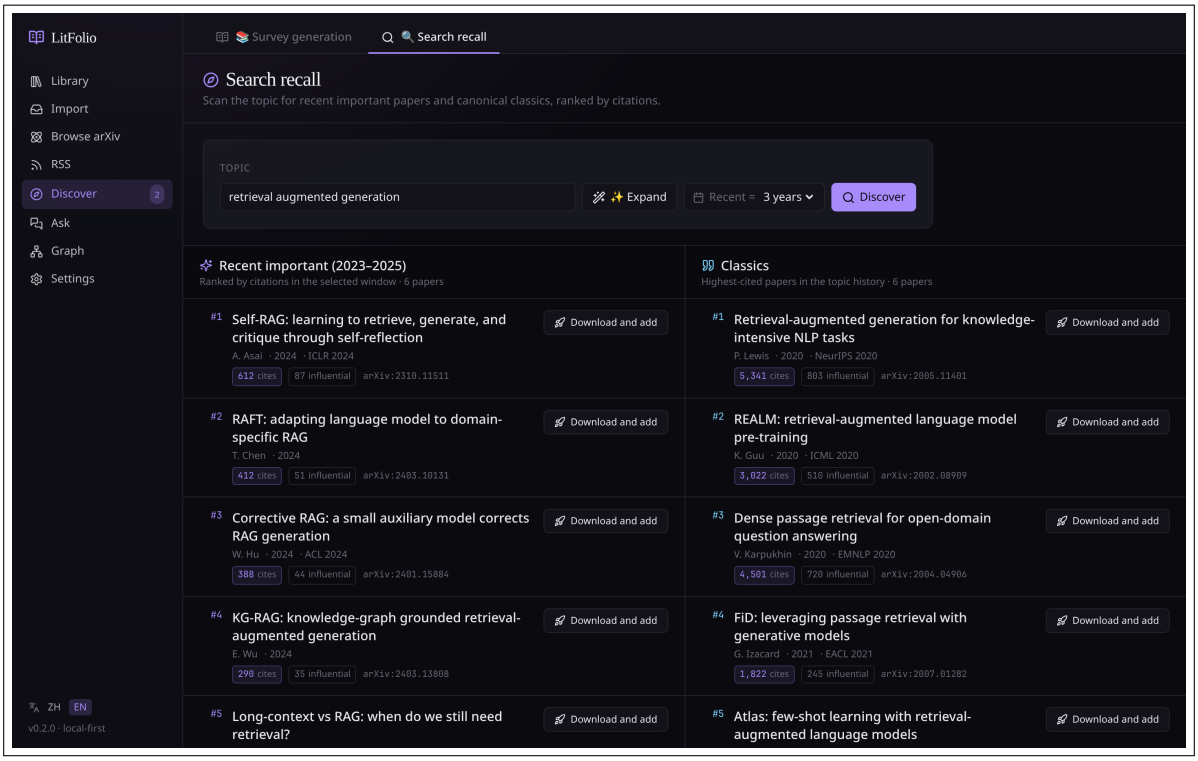


**Figure 7.1** Search Recall tab. After entering a query, LitFolio expands it into several related phrases, retrieves them via multi-path FTS5, and ranks results by citation count. Clicking a result opens a drawer with the TL;DR.

### 7.2.1 Query Expansion

Entering ”retrieval augmented generation” gets expanded to queries like ”RAG / retrieval-augmented / vector retrieval / chunk + retrieval / Lewis 2020...” and so on—a dozen or so related queries. This step uses the LLM configured for the `search` task, with the goal of recalling papers that use different phrasing to describe the same concept.

### 7.2.2 Query Expansion Algorithm Details

Expansion is handled by the `query_expand` module. It does not pass the user’s raw input directly to S2; instead, it first asks the LLM to translate the (possibly Chinese) broad topic into 2–4 precise English search terms. The core constraints are written in the system prompt:

- Output only 2–4 search terms—fewer is better than more, as multi-term OR queries only inflate noise.
- Choose terms that domain experts would actually type, rather than piling on keywords from adjacent sub-fields.
- If two candidate terms would recall the same set of papers (e.g., ”attosecond pulses” and ”attosecond laser”), keep only the more standard one.
- Chinese input is translated to English before searching.
- Remove vague modifiers (research / study / novel / recent, etc.).

The LLM returns JSON: `{"terms": [...]}`. The parser has three-level fault tolerance: it first attempts direct JSON parsing; on failure, it extracts the first `{...}` substring (handling markdown fences and conversational wrappers); as a last resort, it splits by comma or newline. At most 4 terms are used.

### 7.2.3 Time Window and Sorting

Results are sorted by citation count by default; the dropdown in the top-right corner allows switching to ”Newest,” ”Relevance,” or ”Must-Read First.” The time window (last 1/3/5 years) filters out older papers.

## 7.3 Survey Generation

### 7.3.1 Pipeline

Survey generation is a four-step pipeline, with each step’s progress pushed to the frontend in real time via IPC events.

#### Why Not Let the LLM List Papers Directly

If you ask an LLM to output a paper list directly, it will fabricate DOIs, authors, and years—hallucination rates are extremely high. LitFolio’s approach strictly separates ”planning” from ”retrieval”: the LLM is only responsible for planning (decomposing sub-fields + providing search terms). The actual paper retrieval from Semantic Scholar is done by program code, and the LLM then annotates the real papers.

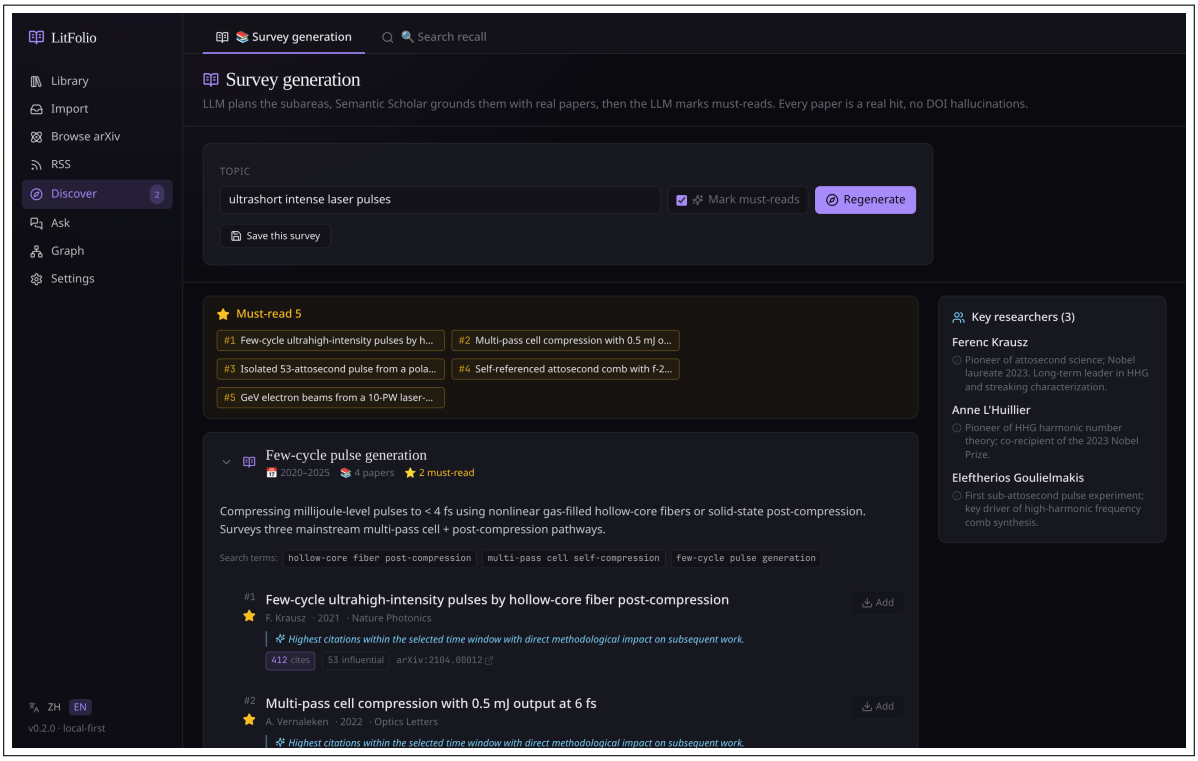


**Figure 7.2** Survey Generation tab. Enter an open topic, and LitFolio uses an LLM to decompose it into 3 sub-fields, fetches 4 representative papers per sub-field from Semantic Scholar, then has the LLM mark ”Must-Read” with a one-line justification.

1. **Planning**—Decompose the open topic into 3 sub-fields (e.g., ”extreme ultrashort pulse laser” might decompose into chirped pulse amplification / harmonic generation / optical parametric processes).
2. **Grounding**—For each sub-field, call the Semantic Scholar Graph API to fetch 4 highly cited papers. These are real references, not LLM fabrications.
3. **Annotating**—The LLM marks each paper as ”Must-Read” with a one-line justification (explaining why this paper is important for the sub-field).
4. **Done**—Save the complete survey to the SQLite `topic_surveys` table for later retrieval.

### Phase 1: LLM Planning

The LLM receives a (possibly Chinese) research topic and outputs a JSON structure: 4–7 sub-fields, each containing a name, active year range, a Chinese narrative, 2–4 English S2 search terms, and an optional PI name hint. Additionally, 5–10 cross-sub-field key_PIs (important researchers) are included. Chinese input is translated into English search terms within the prompt.

### Phase 2: Semantic Scholar Grounding

For each sub-field, the program executes the following steps:

1. **Term-based retrieval**—Each search_term triggers one call to the S2 `bulk_by_citations` API, returning papers sorted by citation count with year filtering. Each term fetches `topk` $\times$ `3` (at least 15) results to ensure sufficient candidates after deduplication.

2. **PI supplementary retrieval**—Combine PI names with search_terms to query S2; each PI is combined with at most 2 search terms.
3. **Deduplication**—Within each sub-field, deduplicate by paperId (prefer paperId; use DOI if paperId is unavailable). When duplicates exist, keep the version with higher citation count.
4. **Relevance filtering**—The paper title and abstract must contain at least 2 stop-word-removed search term tokens, or directly contain a complete search phrase. S2 occasionally returns irrelevant papers; this filtering layer specifically catches them.
5. **DOI filtering + sorting**—Papers without a DOI are discarded (to ensure traceability). The remaining papers are sorted by citation count in descending order and the top-K are selected.

Cross-sub-field deduplication uses a ”first come, first served” strategy: if a paper matches in multiple sub-fields, it is kept only in the sub-field processed first.

#### S2 API Rate Control

LitFolio has a built-in token bucket rate limiter: at most 80 requests per 300-second window (S2’s unauthenticated limit is 100 requests per 5 minutes; LitFolio leaves a 20% margin). It is implemented with a global atomic counter that resets automatically when the window expires. When the limit is exceeded, a clear error message ”try again later” is returned instead of silent failure. During survey generation, multiple search_terms within a sub-field are called serially. If there are many sub-fields and search terms, rate limiting may be triggered—this is why survey generation sometimes takes 30–60 seconds.

#### Phase 3: LLM Annotation (Optional)

The real paper list from all sub-fields (id + title + year + abstract) is sent to the LLM with the following requirements:

- Write 1–2 sentences in Chinese explaining ”why this matters” for each paper.
- Mark 8–12 papers across all sub-fields as ”Must-Read,” ensuring coverage across different sub-fields (do not cluster them in a single sub-field).

Phase 3 is optional—if this LLM call fails, the survey is still usable; it simply lacks annotations and must-read markers.

### 7.3.2 Saving and Restoring

Each generation is automatically saved. The ”History” button lets you return to a previous survey without re-calling the LLM or APIs.

> **Tip** Chinese input is fully supported. LitFolio translates the Chinese topic into English search terms during the planning step before querying S2, so ”extreme ultrashort pulse laser” becomes something like ”ultrashort intense laser pulses” for literature retrieval. The entire process typically takes 30–60 seconds.

## 7.4 Topic Alerts

Topic Alerts are an automated extension of Topic Discovery—once you set a research direction, LitFolio periodically monitors for newly published relevant papers. Unlike RSS subscriptions, which track **sources** (an arXiv category or journal), Topic Alerts track **topics** (cross-source research directions).

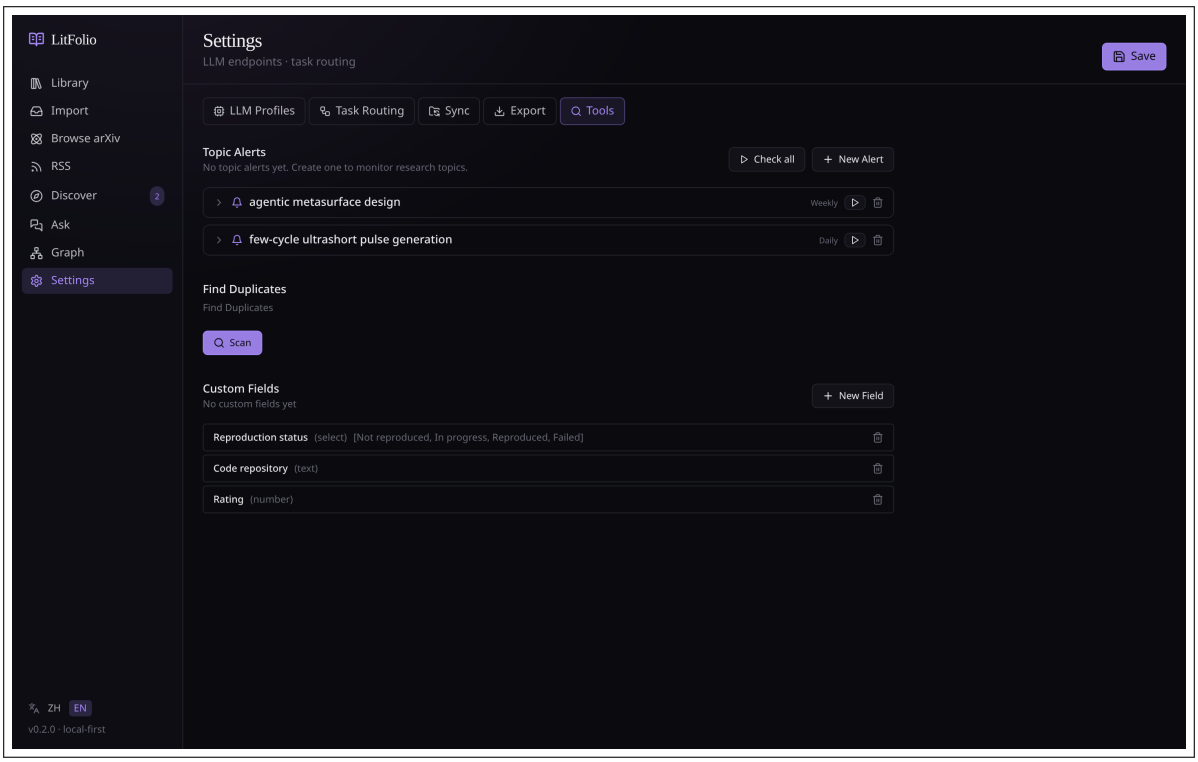


**Figure 7.3** Topic Alert management interface. Set the query, frequency, and target folder; the system runs automatically and reports newly discovered papers.

### 7.4.1 Creating an Alert

Creating an alert requires the following settings:

**Query** A natural language description of the research topic (same input format as Topic Discovery), e.g., ”chirped pulse amplification for extreme ultrashort pulse lasers.”

**Frequency** Daily / Weekly / On startup.

**Target Folder** Optional—papers imported automatically will be placed in the specified folder.

**Auto Import** When enabled, newly discovered papers are automatically imported into the library.

### 7.4.2 Alert Execution Flow

When an alert triggers, LitFolio executes the following flow:

1. **LLM Planning**—Translate the query into 2–4 English search terms (using the same logic as Topic Discovery’s `query_expand`).

2. **Semantic Scholar Retrieval**—Call the S2 API for each search term, fetching the latest papers sorted by citation count.
3. **Deduplication Filtering**—Compare the retrieval results against two lists:
   - Already-imported papers (DOI / arXiv ID match) → excluded.
   - Papers shown in previous alerts (`topic_alert_results` table) → excluded.

   Only papers that are **neither imported nor previously shown** are kept.
4. **Store Results**—New papers are written to the `topic_alert_results` table with `seen = 0`.
5. **Notification**—The topic entry in the navigation bar displays an unread count badge.

### 7.4.3 Auto Import

If "Auto Import" is enabled, the following additional steps execute after step 4:

5. **Batch Import**—For each new paper, invoke the arXiv/DOI import workflow to fetch metadata and PDF.
6. **Place in Target Folder**—If a target folder is specified, imported papers are automatically placed into that folder.

### 7.4.4 Management and Viewing

Manage all alerts in Settings → Topic Alerts:

- View the history of results for each alert.
- Mark results as read (`seen = 1`).
- Edit an alert's query, frequency, or target folder.
- Pause or delete an alert.

> **Warning** Auto Import consumes LLM tokens (for metadata translation, etc.). If you have multiple high-frequency alerts with Auto Import enabled, consider binding the related tasks to a low-cost model.

Chapter 8

# Library Q&A (RAG)

The `/ask` route turns LitFolio into **your personal knowledge base** for asking questions. Unlike generic ChatGPT, answers are based solely on papers you have already imported, and every factual claim is accompanied by a `[N]` citation reference.

## 8.1 How It Works

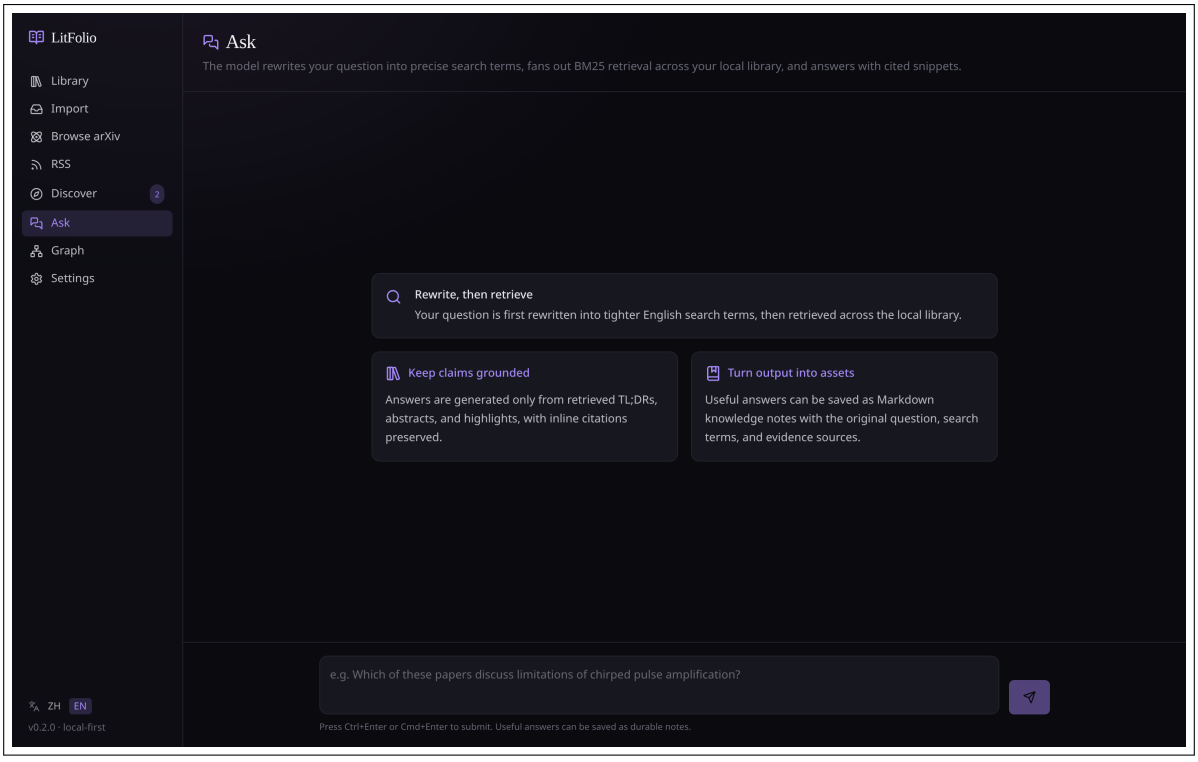


**Figure 8.1** Ask page initial state. Three cards explain the workflow: 1) question rewriting; 2) multi-path FTS5 retrieval; 3) LLM answer with citations.

The entire process has four steps:

1. **LLM Question Rewriting**—This step is the key to the entire RAG pipeline. Throwing a Chinese question directly into FTS5's AND-of-tokens path yields essentially zero recall (FTS5 `unicode61` tokenizes Chinese character by character, so "neural network" becomes "neural AND network" in individual characters). Therefore, the `query_expand` module first rewrites the natural language question into 2–4 English search terms. If rewriting fails (offline, invalid key, model not configured), it does not block the flow—it degrades to searching with the original question directly.
2. **Multi-Path FTS5 Retrieval + Merged Scoring**—Each search term runs an independent FTS5 BM25 search (each term over-fetches `limit` $\times$ `3`, at least 8 results), then

results are merged by paper_id. The sorting rule during merging is: **number of matched terms** (papers matched by more search terms rank higher) > descending year > descending import time. This way, papers that are "relevant from multiple semantic angles" take priority over papers that score high on only one term. If all expanded queries return empty, a fallback FTS5 search using the original question is performed.

3. **Evidence Assembly**—Each recalled paper is compressed into a snippet of no more than 1800 characters, assembled by priority: TL;DR → Abstract → Problem → Method → Comparison → Limitations → Key Findings → User highlights (up to 3, each 240 characters). All snippets combined do not exceed 14,000 characters; excess is truncated.
4. **LLM Answer**—The system prompt enforces: answer only based on the provided evidence, annotate each fact with a `[N]` citation reference, explicitly state "no relevant literature found" when evidence is insufficient rather than guessing, and reply in Chinese. If the retrieval results are empty, a fixed prompt message is returned without calling the LLM.

## 8.2 Multi-Turn Chat Interface

The Ask page has been upgraded to a **chat-style conversational interface** that supports follow-up questions. Each turn displays the user's question and the AI's answer, with distinct avatar indicators (blue for user, green for AI).

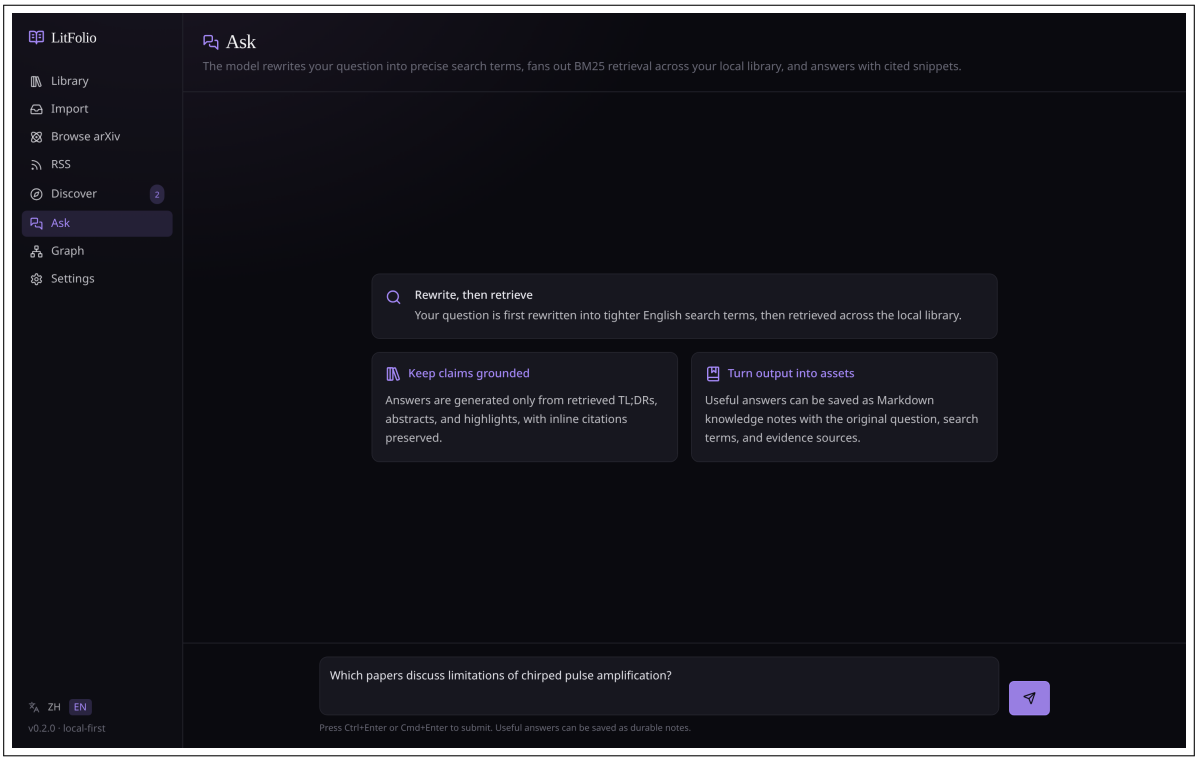


**Figure 8.2** Ask multi-turn conversation. User and AI messages alternate in bubbles, each answer accompanied by source citations and a save button.

### 8.2.1 Conversation State Model

The frontend maintains a `ConversationTurn[]` array, where each element contains:

- `role`—`"user"` or `"assistant"`.
- `content`—The message text.
- `result`—Only present for the assistant role, containing the complete `AskLibraryResult` (answer, sources, model, prompt_tokens, completion_tokens, terms).

Conversation history is **maintained only within the current session**—it is cleared when the page is closed or the route is changed. This is consistent with the single-turn behavior (previous single-turn results were also not persisted across sessions).

### 8.2.2 Multi-Turn RAG Flow

Each follow-up question is processed using the exact same pipeline as a single turn (question rewriting → FTS5 retrieval → evidence assembly → LLM answer). The difference lies in how the LLM message array is constructed:

1. `System prompt`—The same citation-strict system prompt as single-turn.
2. `History messages`—Previous conversation history (all user/assistant message pairs) is inserted as a `ChatMessage` array between the system prompt and the current question.
3. `Current question + evidence`—The current follow-up text plus the newly retrieved Sources block for this turn.

This means:

- **Context carry-over**—The LLM can see previous Q&A, so pronouns and references are correctly resolved. For example, "what method did it use?"—the "it" can refer to the paper discussed in the previous turn.
- **Independent retrieval per turn**—Each follow-up re-runs question rewriting and FTS5 retrieval rather than reusing evidence from the first turn. This ensures that follow-ups can cover new semantic angles.
- **Independent citations**—The `[N]` numbering in each answer corresponds only to that turn's Sources and does not conflict with numbering from previous turns. Source cards are displayed independently below each answer.

### 8.2.3 Conversation History Passing Details

The `conversation_history` sent to the backend is a `Vec<ConversationMessage>`, where each element has `role` and `content`. The backend converts it into a `ChatMessage` array and passes it to the LLM. Note: the history messages **do not contain** Sources blocks—Sources are only attached to the current follow-up's user message. The reasons are:

- Avoid token explosion—if each turn retained full Sources blocks, the context could exceed the model window after 5 turns.
- The LLM only needs the current turn's evidence to answer the current question—previous evidence is already reflected in previous assistant answers.

### 8.2.4 UI Interaction

- The input box is at the bottom of the page; `Ctrl+Enter` / `Cmd+Enter` sends the message.
- The placeholder text changes to ”Continue the conversation...” when conversation history exists.
- After sending a new message, the view automatically scrolls to the bottom (`scrollIntoView` + smooth behavior).
- The ”Clear Conversation” button at the top resets the entire conversation history and input box.
- During loading, the AI avatar + spinning icon + ”Searching...” text are displayed.
- Error messages appear as red text below the input box.

> **Tip** Multi-turn conversation quality depends on the context window size. If your LLM is configured with a small context window (e.g., 4K tokens), conversations beyond 5 turns may truncate history. A model with 8K+ window is recommended.

## 8.3 Save as Knowledge Note

Each answer has a ”Save as Note” button below it. Clicking generates an independent Markdown document (at `notes/ask/<timestamp>.md`), fully preserving the **question, search terms, answer, and evidence sources**. This note is synced along with the library during WebDAV sync.

> **Tip** RAG answer quality depends directly on your library’s coverage—if there are no relevant papers in your library, the answer will honestly state ”no obviously relevant literature found.” This is a **feature**: it will not fabricate references to papers you have not read. In multi-turn conversations, if the first turn finds no literature, subsequent follow-ups will not fabricate references either.

Chapter 9

# Knowledge Graph

The `/graph` route is LitFolio's literature relationship visualization hub. It aggregates citation relationships, methodological inheritance, comparisons, and other connections between papers—as well as associations between papers and conceptual terms—into an interactive knowledge graph.

## 9.1 Network Graph

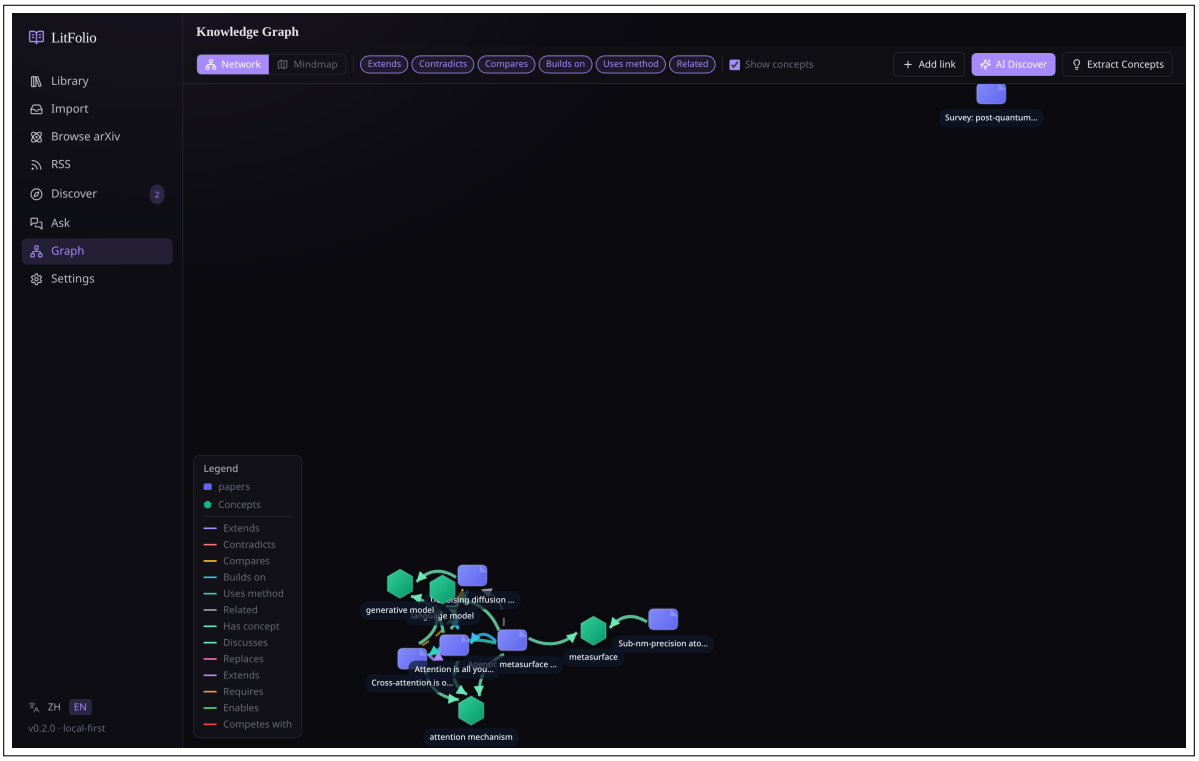


**Figure 9.1** Knowledge Graph—network view. Blue nodes are papers, green nodes are concepts; line colors indicate relationship types; dashed lines represent AI-discovered potential associations.

The network graph is the default view, using force-directed layout to automatically arrange nodes:

- **Paper nodes** (blue)—Fixed size; hover to display title and year.
- **Concept nodes** (green)—Derived from `normalized_term` entries in the term table shared by two or more papers; the node label shows the number of associated papers.
- **Lines**—Color-coded by relationship type (see legend); solid lines are user-created, dashed lines are AI-discovered.

Clicking any node displays its detailed information and a list of all associated nodes in the right sidebar.

## 9.2 Mind Map

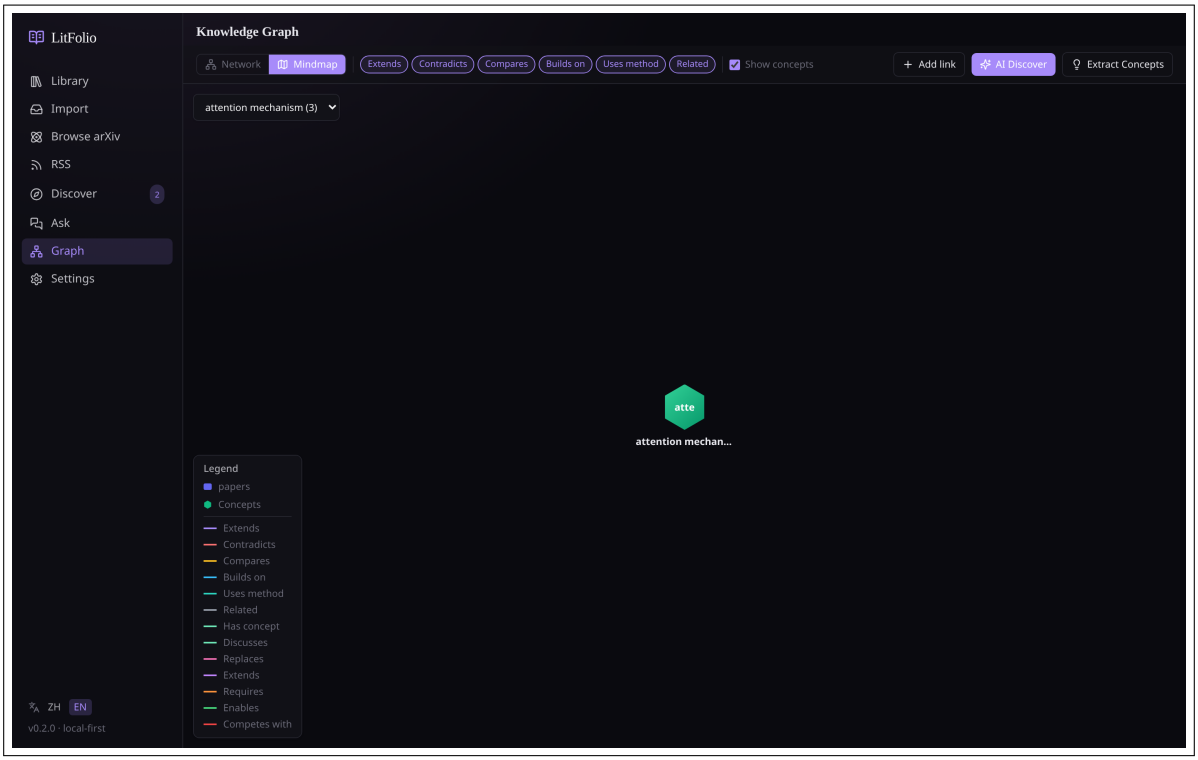


**Figure 9.2** Knowledge Graph—mind map view. A concept is at the center, radiating outward; the first ring contains associated papers; the second ring contains papers linked to those papers.

The mind map takes a conceptual term as its root node and radiates outward in concentric rings:

- **First ring**—Papers directly associated with the concept through the term table.
- **Second ring**—Other papers that have `paper_links` relationships with the first-ring papers.
- Clicking the ”Locate” button on a concept node switches to a mind map centered on that concept.

## 9.3 Manual Relationship Creation

The ”Add Link” button on the toolbar opens a creation dialog. You need to select:

1. **Source paper**—Defaults to the currently selected paper node.
2. **Target paper**—Selected from a dropdown list.
3. **Relationship type**—Six predefined types: `extends`, `contradicts`, `compares`, `builds_on`, `uses_method`, `related`.

4. **Description** (optional)—A brief text describing the specific connection between the two papers.

User-created relationships have a fixed confidence of 1.0 and are displayed with solid lines.

## 9.4 Citation Network

Beyond user-created and AI-discovered relationships, LitFolio can also fetch academic citation relationships from Semantic Scholar—showing "who the academic community considers related to this paper." Click the "Citations" button in the paper drawer or on the Knowledge Graph page.

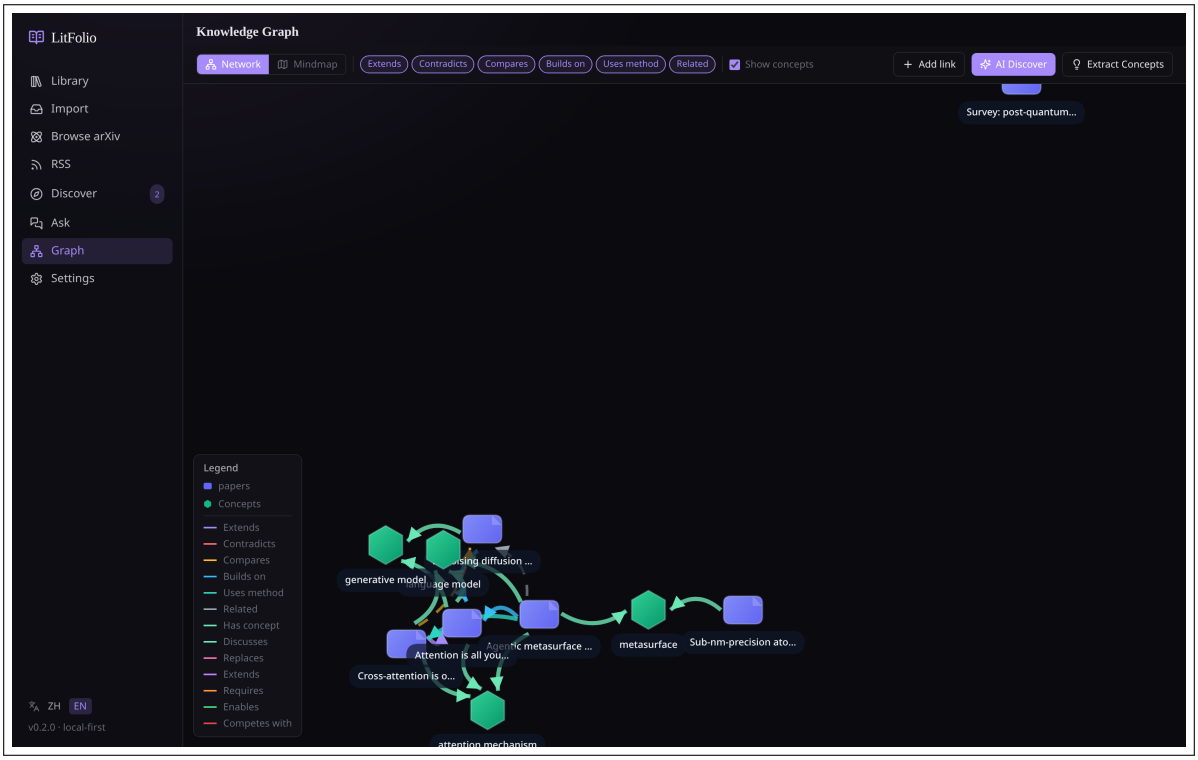


**Figure 9.3** Citation network view. Papers cited by the central paper appear on the left; papers citing it appear on the right. Papers already in the library are highlighted with a green border.

### 9.4.1 Data Fetching Flow

Citation data is obtained through the Semantic Scholar Graph API:

1. **Paper matching**—Send a matching request to S2 using the paper's DOI or title. DOI is preferred (exact match); when no DOI is available, title fuzzy matching is used, and S2 returns the most relevant paperId.
2. **Fetch citation relationships**—Call `GET /paper/{id}?fields=references,citations`, which returns two arrays: `references` (papers this paper cites) and `citations` (papers that cite this paper). Each element contains title, authors, year, venue, and externalIds.
3. **Caching**—Results are written to the `paper_citations` table, with each record containing the direction (`references` / `citations`) and a `fetched_at` timestamp. Subsequent views read directly from cache without re-calling the API.

### 9.4.2 Tree Display

The citation network is displayed in a tree layout:

- **Center node**—The current paper.
- **Left branches**—References (papers cited by this paper), up to 2 levels deep (the first level is direct citations; the second level is citations of those cited papers).
- **Right branches**—Citations (papers that cite this paper), also up to 2 levels.
- **Library indicator**—Papers already in your library are highlighted with a green border; hovering shows the TL;DR (if available).
- **Non-library papers**—Title, authors, and year are displayed; clicking shows abstract details with an "Import" button.

### 9.4.3 Relationship with the Knowledge Graph

The citation network and the knowledge graph are complementary dimensions:

**Citation Network** Objective data from the academic community—who cites whom, independent of your judgment.

**Knowledge Graph** Derived from your subjective judgment and AI inference—methodological inheritance, contradictions, comparisons, and other relationships between papers.

The two can be cross-validated: if the knowledge graph shows that A extends B, and the citation network confirms that A indeed cites B, the relationship's credibility is higher.

> **Warning** Semantic Scholar's citation data coverage is not 100%—very recent papers (published less than a week ago) or papers from niche journals may have no citation data. In such cases, the API returns empty arrays, and the interface displays "No citation data available."

## 9.5 Concept Graph

The concept graph is an advanced dimension of the knowledge graph—it visualizes not relationships between papers, but **evolutionary relationships between methodological concepts**. Click the "Extract Concepts" button on the toolbar to start.

### 9.5.1 Concept Extraction Pipeline

After clicking "Extract Concepts," LitFolio executes the following pipeline for each paper:

1. **Text preparation**—Take the paper's TL;DR + abstract + deep-read sections (if available), concatenated into input text of no more than 3000 characters.
2. **LLM extraction**—Pass the text to the LLM; the system prompt requires returning a JSON array where each element contains:
   - `name`—Methodological concept name (e.g., `Transformer`, `LoRA`, `Self-Attention`), with normalized naming (capitalize first letters, avoid abbreviations unless the abbreviation is more commonly used).
   - `description`—A one-sentence Chinese description of the concept.

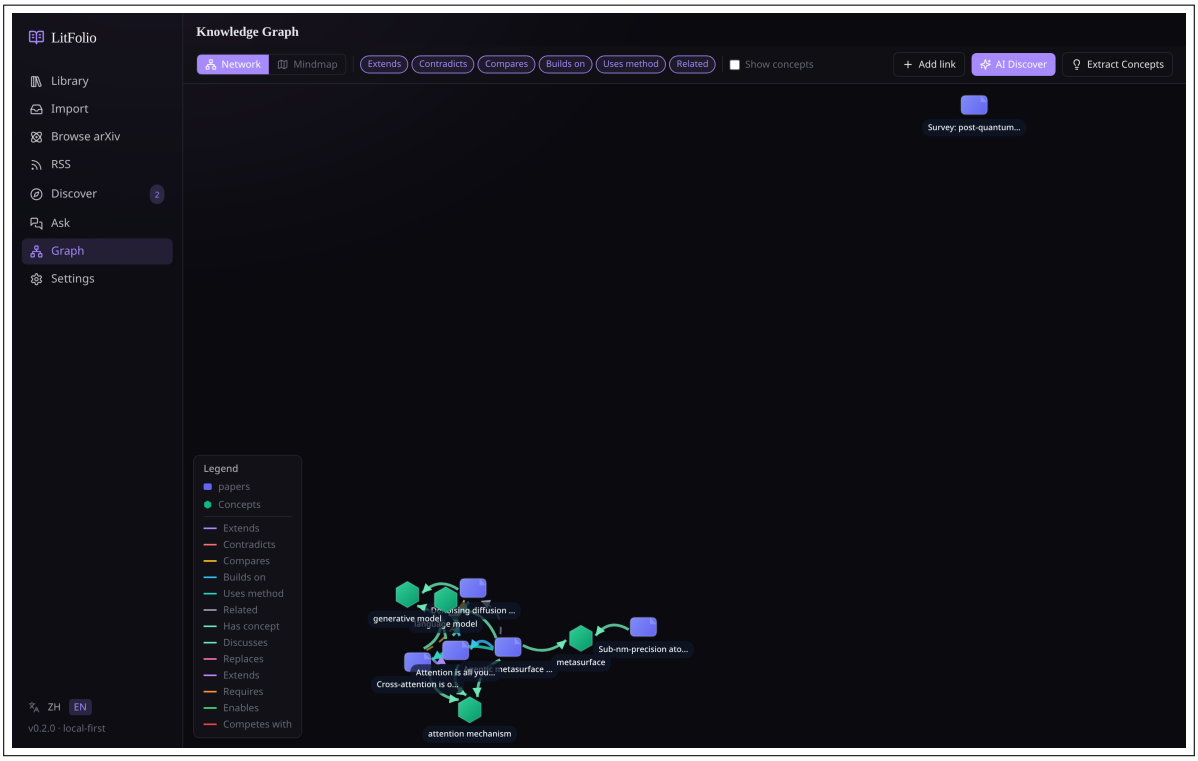


**Figure 9.4** Concept graph. Concept nodes (green) are connected by different colored lines indicating relationship types: replaces (pink), extends (purple), requires (orange), enables (green), competes_with (red).

- `relations`—A list of relationships with other concepts, each containing `target` (target concept name), `relation` (relationship type), and `snippet` (evidence text from the original paper).

3. **JSON parsing**—Three-level fault tolerance (direct parsing → extract `[...]` substring → empty array fallback), consistent with the TL;DR parsing strategy.
4. **Deduplication and normalization**—Convert extracted concept names to lowercase for deduplication comparison. If a concept with the same name already exists in the library (case-insensitive), reuse its existing ID; otherwise, create a new concept.
5. **Relationship creation**—For each relationship, look up the target concept (name-based matching); if the target concept does not exist, create it first. Then create an edge in the `concept_relations` table, recording the `evidence_paper_id` and `snippet`.
6. **Paper-concept association**—Create a `discusses` edge in the `paper_concepts` table with default relevance = 1.0.

### 9.5.2 Concept Relationship Types

Five predefined relationship types, each corresponding to a different academic semantic:

| Type | Meaning | Typical Example |
|---|---|---|
| `replaces` | New method replaces old method | Transformer replaces RNN |

| Type | Meaning | Typical Example |
|---|---|---|
| `extends` | Improvement on existing method | LoRA extends Fine-tuning |
| `requires` | Depends on another method | Attention requires Embedding |
| `enables` | Makes another method possible | GPU enables Deep Learning |
| `competes_with` | Contemporary alternative | BERT competes_with GPT |

### 9.5.3 Graph Visualization Integration

After extraction completes, concept nodes automatically appear in the knowledge graph:

- **Network graph**—Concept nodes are green with fixed size. Relationships between concepts use different colored lines: replaces (pink), extends (purple), requires (orange), enables (green), competes_with (red). Papers and concepts are connected by `discusses` edges (light green).
- **Mind map**—Concepts serve as root nodes radiating outward. The first ring contains papers directly associated with the concept (via `paper_concepts`); the second ring contains papers linked to those papers (via `paper_links`).
- **Legend**—The legend panel in the bottom-left corner annotates all relationship types with their line colors and styles.

### 9.5.4 Manual Management

You can manually correct AI extraction results:

- **Create concept**—Manually add concepts the AI missed.
- **Edit concept**—Modify the name or description.
- **Delete concept**—Upon deletion, associated `paper_concepts` and `concept_relations` records are cascade-deleted.
- **Create/delete relationship**—Manually add or remove relationships between concepts.
- **Associate/disassociate papers**—Manage `discusses` edges between concepts and papers.

> **Tip** Concept extraction quality depends on the quality of the paper's deep-read. If a paper only has a TL;DR without deep-read sections, the LLM has less context to work with, and the extracted concepts may be less precise. It is recommended to run deep-read on papers before performing concept extraction.

## 9.6 AI-Discovered Relationships

Click the "AI Discover" button on the toolbar, and LitFolio will:

1. Collect metadata from all papers in the library (title, year, TL;DR, methods, key findings).
2. Group them by shared terms (up to 12 papers per batch) and send to the LLM to analyze potential associations.
3. Parse the LLM's JSON output and generate relationship suggestions with confidence scores and descriptions for each pair of papers.

AI-discovered relationships are displayed with dashed lines and have confidence scores below 1.0. In the reader's term panel, AI relationships show a confidence percentage. You can:

- **Accept**—Promote the AI relationship to a user relationship (confidence becomes 1.0).
- **Reject**—Delete the relationship suggestion.

## 9.7 Filtering and Legend

The toolbar provides two filter groups:

- **Relationship type**—Filter which lines are displayed by type label.
- **Include concepts**—Toggle concept node visibility. When disabled, only direct paper-to-paper relationships are shown.

The legend panel in the bottom-left corner annotates node colors and the line style for each relationship type.

## 9.8 Reader Integration

At the bottom of the reader's term panel (Chapter 4), all associated papers for the current paper are displayed. Each association shows the relationship type and source (user/AI); clicking navigates to the corresponding paper's reader view.

> **Tip** The knowledge graph data comes entirely from the local SQLite database (intersection of the `paper_links` table and the `paper_terms` table). Relationship data is synced along with the library during WebDAV sync.

Chapter 10

# Similar Paper Recommendations

After reading a paper, the natural question is ”what else is worth reading?” Similar Paper Recommendations use Semantic Scholar’s recommendation API to automatically find related literature.

## 10.1 Usage

Click the ”Similar Papers” button in the paper drawer or at the top of the reader. LitFolio uses the paper’s DOI or title to send a recommendation request to Semantic Scholar.

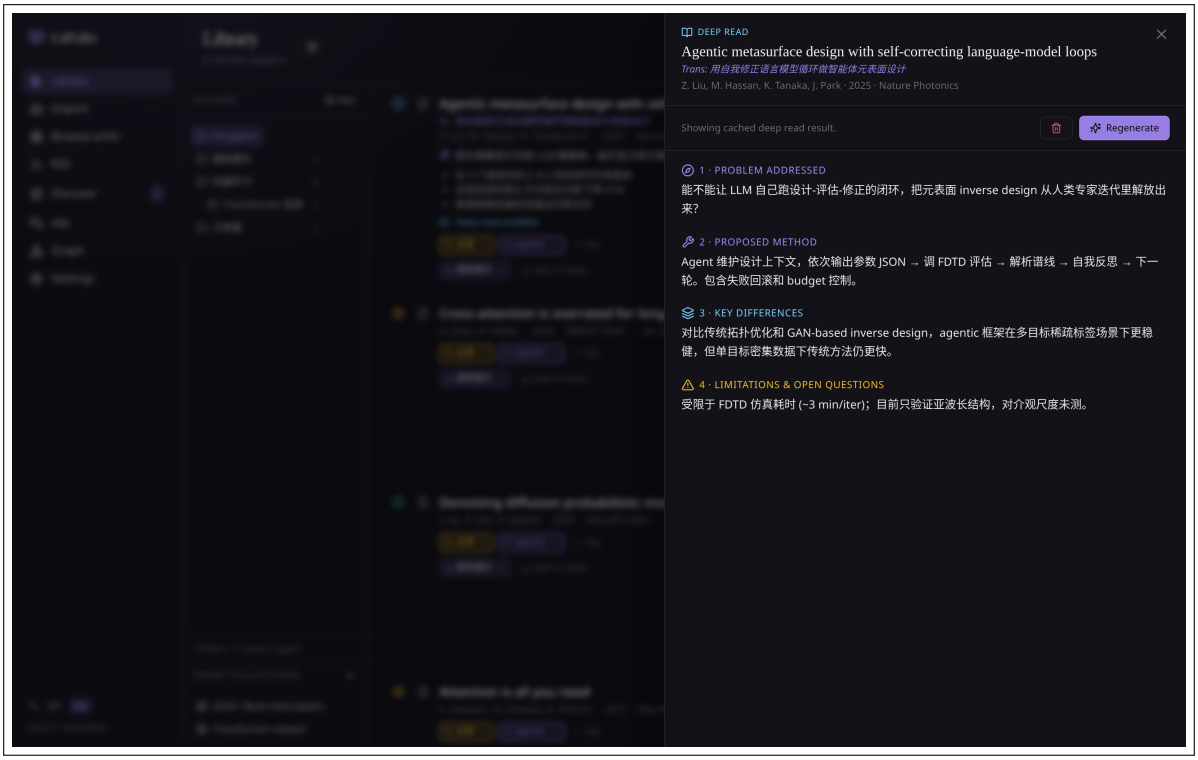


**Figure 10.1** Similar Paper Recommendations panel. Result cards display title, authors, year, abstract snippet, and relevance score.

### 10.1.1 Recommendation Fetching Flow

1. **Paper matching**—Use DOI (preferred) or title to match a paperId with Semantic Scholar. The same matching logic as the citation network is used.

2. **Call recommendation API**—Request `GET /paper/{id}/recommendations`, which returns a list of papers sorted by relevance. Each entry contains title, authors, year, abstract, and citationCount.
3. **Library filtering**—Iterate through the recommendation results and check each against the library using DOI and arXiv ID. Papers already in the library are removed from the results.
4. **Caching**—Filtered results are written to the `recommendation_cache` table with a `fetched_at` timestamp. Results are read directly from cache for 7 days.

### 10.1.2 Result Display

Each result card contains:

- Title, authors (first 3 + `et al.`), year.
- Abstract snippet (first 200 characters).
- Citation count (if S2 returned this field).
- ”Import” button—clicking fetches full metadata and attempts to download the PDF.

### 10.1.3 Edge Cases

- **Papers without DOI**—Title fuzzy matching may return an incorrect paperId, leading to irrelevant recommendation results. In this case, the interface displays ”Match accuracy is low; results are for reference only.”
- **Very recent papers**—S2’s recommendation model requires papers to have some citation accumulation before generating recommendations. Papers published less than a month ago typically have no recommendation results.
- **Niche fields**—S2’s data coverage is not 100%; some non-English or non-mainstream conference papers may not match.

> **Tip** If recommendation results are empty, try using the manual search in ”Topic Discovery” (Chapter 7), or check AI-discovered associations in the Knowledge Graph.

Chapter 11

# Literature Review Draft Generation

After collecting 20–50 papers on a research direction, the first draft of a literature review is the hardest to write. LitFolio's AI review generation helps you build the skeleton.

## 11.1 Usage

Select multiple papers in the library (using checkbox mode) and click "Generate Review" on the toolbar. You can also select "Generate Review" from a folder's right-click context menu.

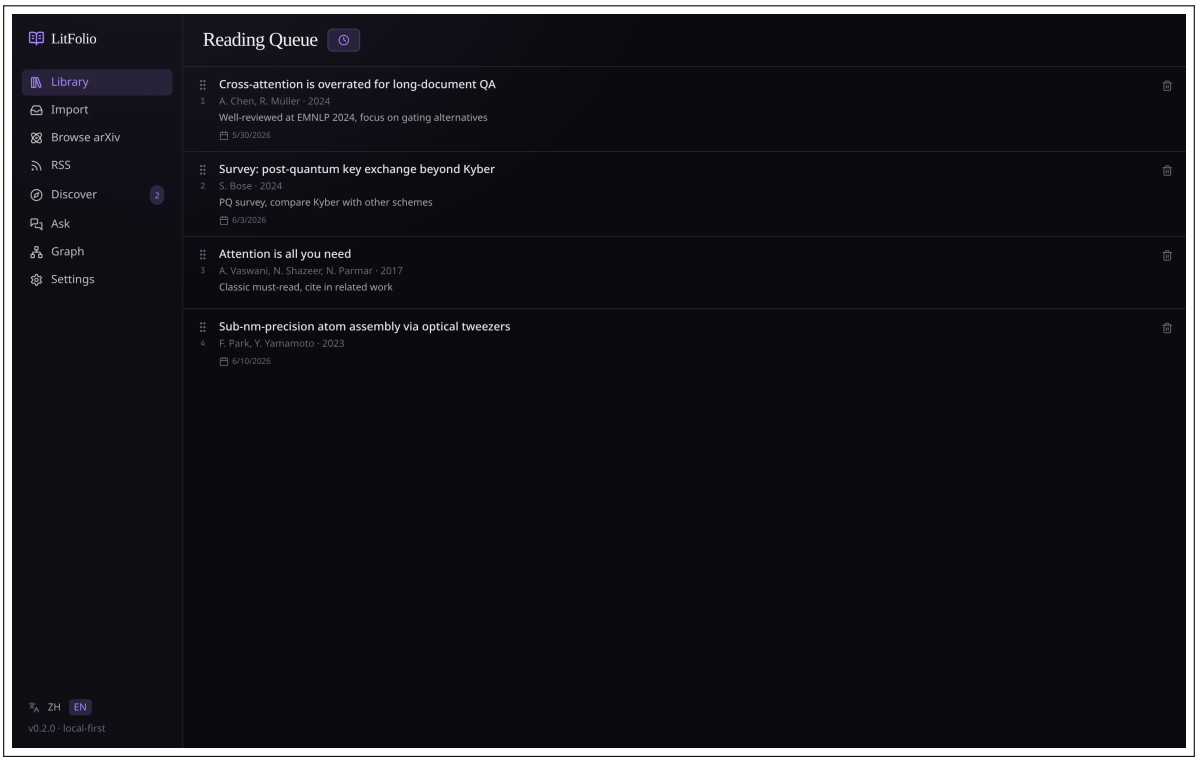


**Figure 11.1** Generated literature review draft. Organized by theme sections, each citing relevant papers, with a research gaps section at the end.

### 11.1.1 Generation Pipeline

Review generation is a two-step pipeline:

**Step 1: Grouping**

Send the selected papers to the LLM and ask it to group them by theme/method/application area. The LLM returns JSON:

```
{
  "groups": [
    {
      "name": "Attention Mechanism Variants",
      "paper_ids": ["p1", "p3", "p7"],
      "rationale": "These papers all improve attention computation efficiency"
    },
    {
      "name": "Positional Encoding Methods",
      "paper_ids": ["p2", "p5"],
      "rationale": "Addressing Transformer position awareness"
    }
  ]
}
```

Grouping strategy options: `method` (by method, default), `year` (by era), `application` (by application area). Different strategies produce different groupings.

**Step 2: Writing**

Concatenate the grouping results with each paper's TL;DR + deep-read sections + abstract into a prompt, and ask the LLM to write the review. System prompt constraints:

- Each group corresponds to a second-level section.
- Every paper is cited at least once in the text, using the format `[Author, Year]`.
- The introduction outlines the research background and the review's scope.
- A "Research Gaps" section at the end identifies unresolved problems in the current literature.
- The conclusion summarizes main findings and future directions.
- Language is academic Chinese, consistent with the style of formal review papers.
- Do not fabricate papers or data not provided.

### 11.1.2 Input Materials

Materials provided to the LLM for each paper (by priority, total tokens not exceeding the window limit):

1. TL;DR + key findings (approx. 100 tokens/paper)
2. Deep-read sections: Problem / Method / Comparison / Limitations (approx. 400 tokens/paper)
3. Abstract (approx. 200 tokens/paper)

Input for 20 papers is approximately 14,000 tokens; including the system prompt and output space, a model with at least a 16K context window is required.

## 11.2 Output and Editing

- Output is in Markdown format, 2000–5000 words (for 20 papers).
- Can be edited directly in the interface to correct AI summaries or supplement your own observations.
- Can be saved as a note file (`notes/review/<timestamp>.md`).
- Can be regenerated—choose a different grouping strategy (by method / by year / by application area).

> **Warning** The review draft is a starting point, not a finished product. The AI may overlook subtle differences between papers or overgeneralize. Be sure to manually review and supplement with your own analysis. Pay particular attention to whether the ”Research Gaps” section covers the directions you consider important—the AI may over-emphasize certain papers while neglecting others.

# Part III

# Advanced Tools and Administration

Chapter 12

# Multi-paper Comparison

After selecting 3–5 papers on the same topic, AI can generate a structured comparison matrix to help you quickly sort out differences in methods, datasets, metrics, and conclusions.

## 12.1 How to Use

Select papers in the library (at least 2), then click “Compare” in the toolbar. LitFolio will call the LLM to perform a structured analysis of each paper’s TL;DR and deep-read content.

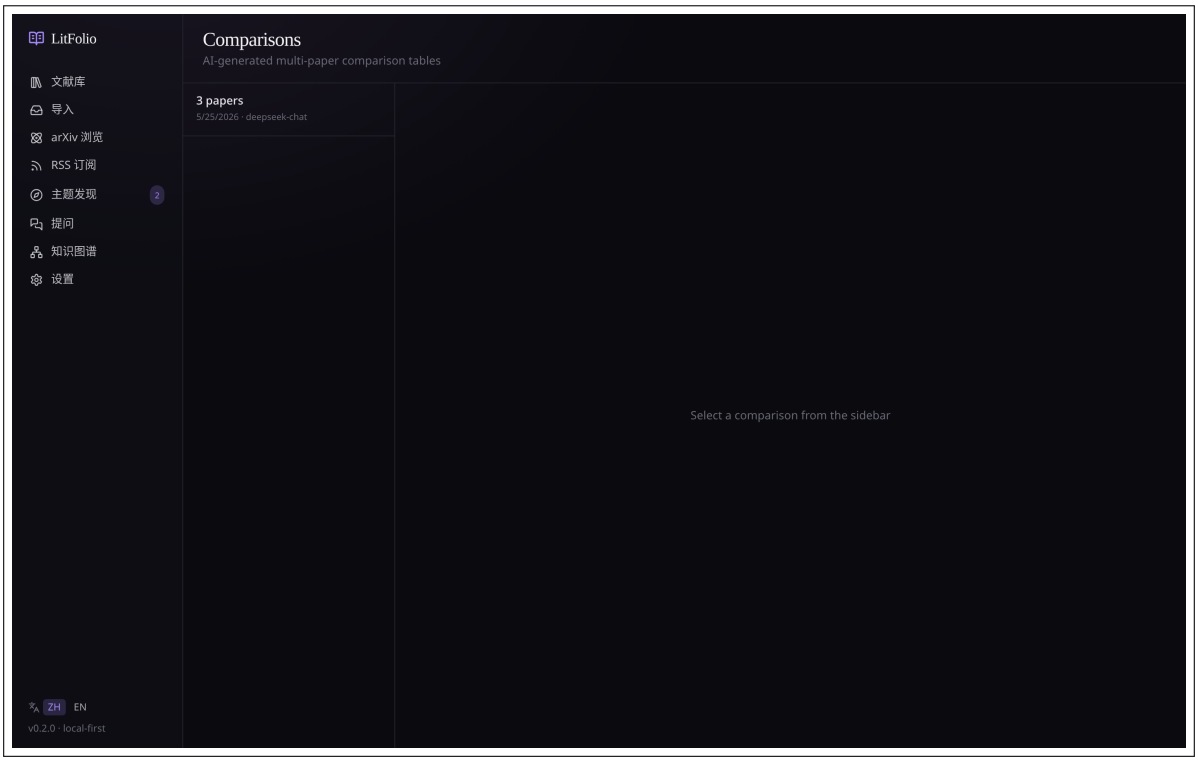


**Figure 12.1** Multi-paper comparison page. The table is organized into six columns: Problem / Method / Dataset / Key Metric / Limitation / Conclusion. Cells are directly editable.

### 12.1.1 Comparison Generation Pipeline

1. **Material Collection**—for each selected paper, query its TL;DR, key findings, deep-read four-section content (Problem / Method / Comparison / Limitations), and abstract. If a paper has no deep-read results, only TL;DR + abstract are used.

2. **Prompt Construction**—concatenate all paper materials in a fixed format, separated by `=== PAPER: <title> ===` for each paper. The system prompt instructs the LLM to return a JSON array, where each element corresponds to one paper and contains six columns of content.
3. **JSON Parsing**—three-level fault-tolerant parsing (consistent with TL;DR). If parsing fails, an attempt is made to extract data from the LLM's raw text in table format.
4. **Storage**—results are written to the `paper_comparisons` table. The `paper_ids` field is stored as a JSON array, and the `content` field stores the Markdown table text.

### 12.1.2 Comparison Table Structure

The comparison results are presented in a table with the following columns:

| Column | Content |
|---|---|
| Paper | Paper title |
| Problem | The problem addressed (2–3 sentences) |
| Method | Core method (focus on innovations) |
| Dataset | Datasets used and their scale |
| Key Metric | Primary metric name and value (e.g., `BLEU 41.2`, `F1 0.89`) |
| Limitation | Limitations (1–2 sentences) |
| Conclusion | Conclusion (1 sentence) |

### 12.1.3 Editing and Saving

- The table supports **inline editing**—click any cell to modify the AI output. Edited content is automatically saved to the database.
- Comparison results persist across sessions—no need to regenerate on next open.
- The "Regenerate" button reruns the AI with the latest paper content (overwriting existing results).
- Comparing 5 papers typically completes within 30 seconds.

> **Tip** Comparison quality depends on the deep-read quality of each paper. If a paper has not been deep-read, only the TL;DR and abstract will be used, resulting in less information. It is recommended to run a deep read on all papers before comparing.

Chapter 13

# Markdown Export

LitFolio's notes and highlights can be exported as structured Markdown files, compatible with Obsidian, Logseq, and general Markdown workflows.

## 13.1 Export Content

Each paper generates a single `.md` file with the filename format: `<first_author>_<year>_<first_word_of_` Special characters in filenames are replaced with underscores.

### 13.1.1 File Structure

```
---
title: "Attention Is All You Need"
authors: ["Vaswani, Ashish", "Shazeer, Noam"]
year: 2017
venue: "NeurIPS"
doi: "10.48550/arXiv.1706.03762"
arxiv_id: "1706.03762"
tags: ["transformer", "attention"]
folders: ["ML/Transformer"]
read_status: "read"
bibtex: "@inproceedings{vaswani2017attention, ...}"
custom_project: "thesis-ch3"
custom_priority: "high"
---

## Notes

Free-text note content (from note.md).

## Structured Notes

### Problem
What problem is being solved...

### Method
What method is proposed...

### Key Numbers
Key experimental data...

### Limitations
```

```
Limitations and open problems...

### Thoughts
Your own reflections...

## Highlights

### Key Finding
- **[p.3]** The self-attention mechanism connects all
  positions with a constant number of operations.
  > User comment (if any)

### Method
- **[p.5]** Multi-head attention allows the model to
  attend to information from different representation
  subspaces at different positions.

## Terms

- **Transformer**: A sequence-to-sequence model based
  on the self-attention mechanism, abandoning recurrent
  and convolutional structures.
  - Evidence: "We propose a new simple network
    architecture, the Transformer."
  - Cross-paper reuse: 3
```

### 13.1.2 Frontmatter Fields

The YAML frontmatter contains all metadata fields for the paper. Custom metadata fields are written in the format `custom_<field_name>`. Tags and folders are output as arrays, making it easy to query with Obsidian's Dataview plugin.

### 13.1.3 Highlights Grouped by Label

Highlights are grouped by annotation color label (see Section 4.6). Each label serves as a third-level heading, with its highlights sorted by page number. Each highlight includes:

- Page number and color label.
- Highlight text (bold).
- User comment (if any, displayed as a block quote).

### 13.1.4 Terms and Bidirectional Links

Terms are output with each term name wrapped in `[[term name]]` double brackets, leveraging Obsidian's wikilink syntax for graph integration. If a note in the same paper references a term, Obsidian will automatically establish a bidirectional link.

## 13.2 Incremental Export Algorithm

Incremental export is tracked via the `papers.last_exported_at` field:

1. Query `SELECT id FROM papers WHERE last_exported_at IS NULL OR updated_at > last_exported_at`.
2. For each paper, compare `updated_at` (last metadata/note/highlight change time) with `last_exported_at` (last export time).
3. Only export papers where `updated_at > last_exported_at`.
4. After successful export, update `last_exported_at = current timestamp`.

   This ensures that batch exports do not redundantly write unchanged files.

## 13.3 Configuration and Triggers

Configure in "Settings → Export":

- **Export Directory**—e.g., `~/ObsidianVault/LitFolio/`.
- **Incremental Export**—enabled by default. When disabled, all papers are exported each time.
- **Auto Export**—when enabled, saving notes or highlights automatically triggers an export (5-second debounce to avoid frequent writes).

You can also manually click "Export Now" to batch-export all papers. A full export of 500 papers typically completes within 10 seconds.

> **Tip** Obsidian automatically parses YAML frontmatter. You can filter and sort papers in Obsidian's property view by fields such as `year`, `read_status`, and `tags`. Combined with the Dataview plugin, you can create dynamic query views such as "all unread papers published after 2024."

Chapter 14

# Command Palette

The command palette is LitFolio's keyboard-driven quick-action hub. Press `Ctrl+K` (macOS: `Cmd+K`) on any page to open it.

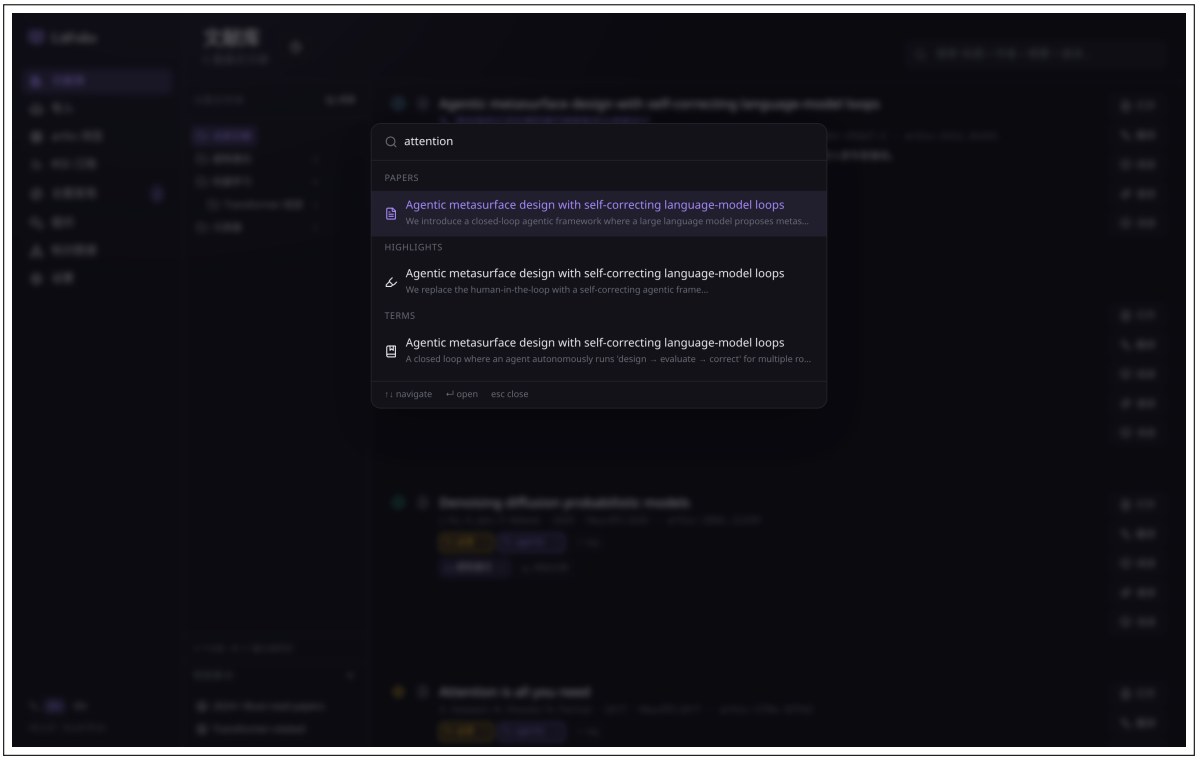


**Figure 14.1** Command palette. The top input field supports fuzzy search, with results grouped into Navigation / Papers / Actions.

## 14.1 Three Modes

The command palette indexes three data sources simultaneously, filtering in real time as you type:

**Navigation** Page route names (`library`, `reader`, `graph`, `settings`, `browse`, `feeds`, `topic`, `ask`, `import`, `compare`). Selecting one navigates to that page.

**Search** Paper titles and author names. Queries the database for the 20 most recent paper titles and performs fuzzy matching against user input. Selecting one opens it in the reader.

**Actions** Executable commands such as `export bibtex`, `generate terms`, `ai discover`, `extract concepts`, `find duplicates`. Selecting one directly executes the corresponding action.

### 14.1.1 Fuzzy Matching Algorithm

The command palette uses a subsequence-based fuzzy matching algorithm rather than simple substring matching. The algorithm flow:

1. **Input Normalization**—convert both user input and all candidates to lowercase and strip non-alphanumeric characters.
2. **Subsequence Matching**—check whether each character of the user input appears in the candidate in order. For example, `grpah` against `graph`: g->g checkmark, r->r checkmark, p->a cross, skip a, p->p checkmark, a->a checkmark, h->h checkmark → match successful.
3. **Scoring**—after a successful match, calculate a relevance score:
   - Consecutive matching characters earn bonus points (`graph` matching `graph` scores higher than matching `generate paragraph`).
   - Earlier match positions earn more bonus points.
   - An exact match earns additional bonus points.
4. **Sorting**—sort by score in descending order and return the top 10 results.

The advantage of this algorithm is its strong fault tolerance—even if the user makes a typo (`librray` → `library`), as long as the characters appear in order, the match succeeds.

### 14.1.2 Result Grouping and Display

Results are displayed grouped by category:

- **Recent**—the 5 most recently used commands and 3 most recently opened papers, displayed at the top by default (no input required).
- **Navigation**—matched page routes.
- **Papers**—matched papers.
- **Actions**—matched commands.

Use the up and down arrow keys to move the selection, `Enter` to execute, and `Esc` to close.

> **Tip** The command palette is designed so that all features are reachable via keyboard. If you are unsure where a feature is, searching in the command palette is usually faster than browsing menus. It also works well as a quick paper-jumping tool—enter a few keywords from the title to find the target paper.

Chapter 15

# Custom Metadata Fields

Different researchers have different organizational needs. Custom metadata fields let you add arbitrary key-value pairs to papers.

## 15.1 Field Management

Define fields in "Settings → Custom Fields." Field definitions are stored in the `custom_field_defs` table, and field values are stored in the `paper_custom_fields` table.

### 15.1.1 Field Types

| Type | Storage Format | UI Control |
|---|---|---|
| `text` | Arbitrary string | Text input field |
| `number` | Numeric string | Number input field, supports sorting comparisons |
| `date` | Unix timestamp | Date picker |
| `select` | Option value string | Dropdown select; options defined as a JSON array in the `options` field |

### 15.1.2 Data Model

- `custom_field_defs`—field definition table. One row per field: `name` (unique), `field_type`, `options` (JSON array of selectable values for select type), `created_at`.
- `paper_custom_fields`—field value table. Composite primary key `(paper_id, field_id)`; each row stores one field value for one paper.

## 15.2 Usage

Once fields are defined, a "Custom Fields" section appears in every paper's detail drawer. You can directly edit each paper's field values. Custom fields support:

- **Filtering**—in the main list's filter bar, you can filter papers by custom field values. For example, filter all papers where `project = thesis-ch3`.
- **Sorting**—number and date type fields support ascending/descending sort.
- **Export**—in Markdown export's YAML frontmatter, custom fields are written in the format `custom_<field_name>`.

- **Global Definitions**—define once, and all papers share the same field definitions. Deleting a field definition cascades to delete that field's values across all papers.

> **Tip** The difference between custom fields and tags: tags are flat, untyped keywords; custom fields are typed, structured data (such as dates, numbers, enums). If you need to sort by "deadline" or filter by "priority," custom fields are more appropriate than tags.

Chapter 16

# Settings

The `/settings` route is LitFolio's "control panel." The main sections are: **LLM Configuration**, **Task Assignment**, **WebDAV Sync**, **Export**, **Custom Fields**, **Annotation Colors**, and **Topic Alerts**.

## 16.1 LLM Configuration

LitFolio is not locked to any single LLM provider. Its design follows the "OpenAI-compatible protocol": you can add any number of endpoint configurations (profiles), with one marked as "active." Each profile contains an API URL, API key, chat model, embedding model (optional), and sampling temperature.

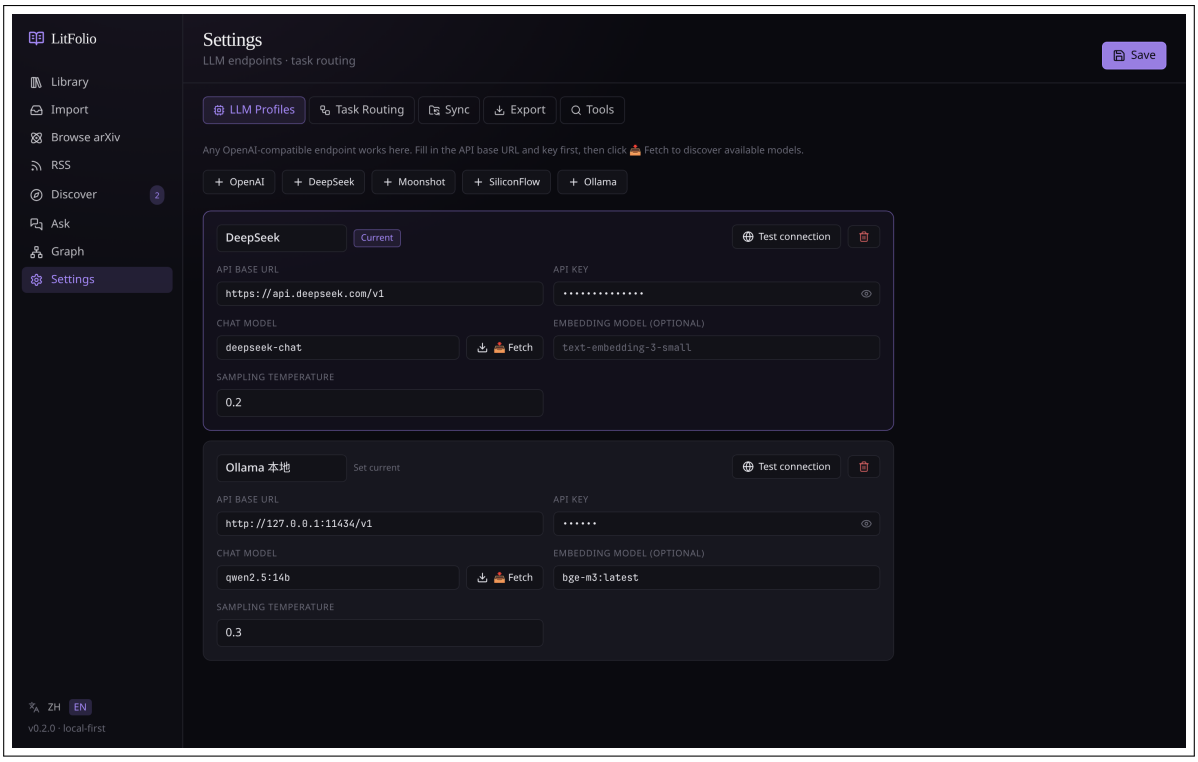


**Figure 16.1** LLM Configuration. Two profiles: DeepSeek and local Ollama; the currently active profile is DeepSeek. Each profile can "Pull" the remote available model list to populate the chatModel field.

### 16.1.1 Pulling Available Models

After filling in the API URL and key, click `Pull`. LitFolio will call the remote `GET /v1/models` endpoint to fetch the available model list. This step prevents manual typos in model names. The returned models populate the dropdown.

### 16.1.2 Testing the Connection

"Test Connection" initiates a small chat request (prompt: "Hello") and verifies the entire pipeline using a real model response. Failure reasons (authentication, network, model not found) are displayed as-is.

## 16.2 Task Assignment

LitFolio groups all LLM calls into **6 tasks**:

- **Quick Read TL;DR**—one-sentence summary + key findings
- **Deep Read quickRead**—four-section deep reading
- **Translation translate**—title/abstract/selection translation
- **Search Expansion search**—query expansion for topic search
- **Survey survey**—topic survey generation
- **Terms term**—glossary extraction
- **Ask ask**—in-library RAG answer

Each task can be independently bound to any profile and any specific model. This lets you configure:

**Expensive but Smart** Deep read, survey, ask → choose **top-tier proprietary chat models** (such as the latest flagships from OpenAI, Anthropic, Google). These affect answer quality, but per-call token usage is modest.

**cheap and Capable** Translation, terms → choose **budget tiers from major domestic API providers** (DeepSeek, Doubao, Zhipu, Moonshot, etc., all offer excellent value). Translation/terms have high call volume, but at a few yuan per million tokens, costs stay minimal.

**Local and Free** TL;DR, search expansion → **open-source chat models on local Ollama** (Qwen, Llama, DeepSeek distilled versions, 7B–32B all work). Zero cost, good privacy, speed depends on your GPU.

### 16.2.1 LLM Call Details per Task

Understanding how each task calls the LLM helps you decide what tier of model to assign.

#### Quick Read TL;DR

Input: title + authors (first 6) + abstract. The prompt requests JSON: `{"tldr": "one sentence, no more than 40 words", "key\_findings": ["3--5 items, each no more than 20 words"]}`. Output language follows user settings. JSON parsing has three-level fault tolerance (direct parse → extract `{...}` substring → empty object fallback). Results are written to the `papers` table cache; clicking again will not re-invoke the LLM.

#### Deep Read Quick Read

Input is the same as TL;DR (title + authors + abstract), but the prompt is more detailed: it requests four-section structured JSON (problem / method / comparison / limitations),

each section 2–4 sentences in prose style, no bullet points allowed. The system prompt positions the LLM as "a senior reviewer helping a colleague decide whether to read the full paper," guiding it to provide judgmental assessments rather than generic summaries.

### Translation

Input: title + abstract. The prompt instructs the LLM to preserve technical terms in their original language (model names, algorithm names, mathematical symbols, units, dataset names), not to paraphrase or summarize, and return JSON `{"title": "...", "abstract": "..."}`.

## 16.3 WebDAV Sync

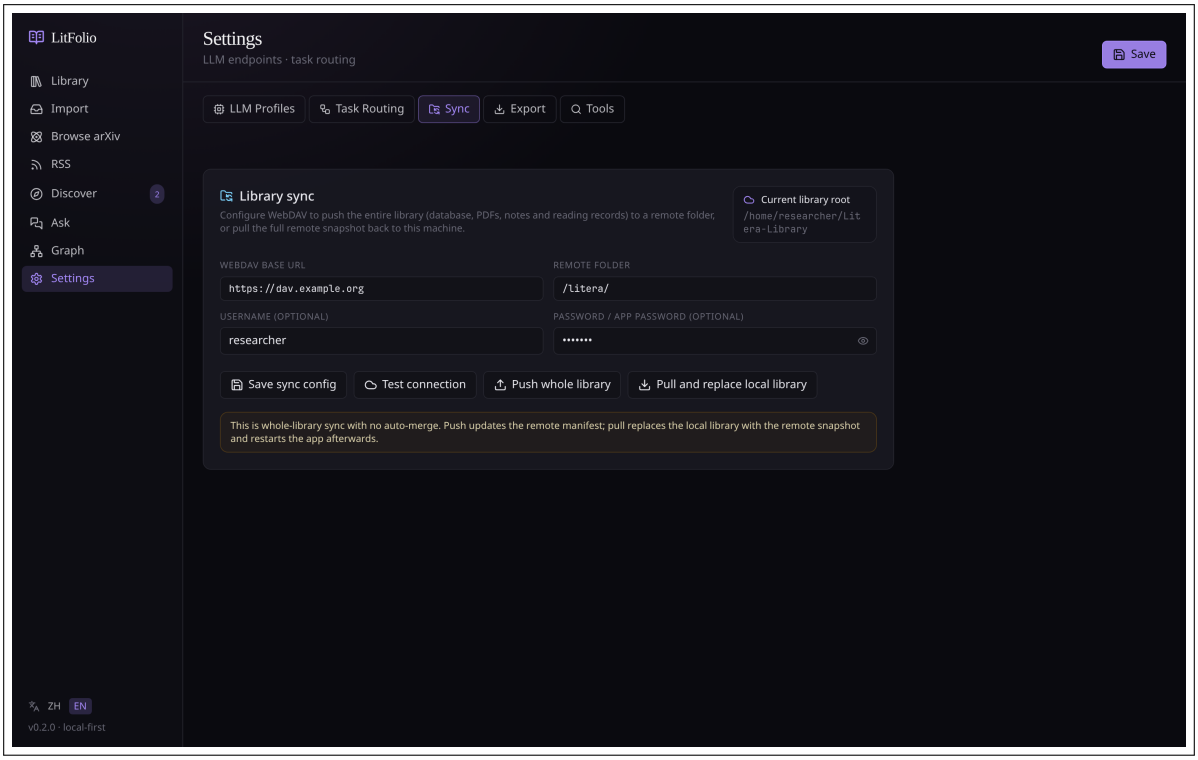


**Figure 16.2** WebDAV Sync panel. After configuring the URL and credentials, you can **Push** (local→remote) or **Pull and Replace Local** (remote→local, which first backs up the local state to `backups/sync/`).

LitFolio's sync uses "full library snapshot" semantics, not fine-grained incremental sync. Specifically:

1. **Push**—packages the entire `~/Litera-Library/` and pushes it to WebDAV, creating a complete copy on the remote.
2. **Pull and Replace Local**—pulls a complete copy from the remote and overwrites the local state.

> **Warning** "Pull and Replace Local" will **overwrite** the local database and all `papers/`. LitFolio automatically saves the current local state to `backups/sync/<timestamp>/` before executing, so you can roll back even if you pull in the wrong direction—but please take this

> operation seriously, especially when both machines have unpushed changes.

The design choice of "full library overwrite" rather than incremental merging is because most modifications to a paper library involve PDFs and notes, and the semantics of which side wins during a conflict are difficult to explain to users; exposing only "push all / pull all" with "automatic backup" is more predictable.

## 16.4 Export Settings

The "Settings → Export" panel controls Markdown export behavior (see Chapter 14 for details):

- **Export Directory**—choose the output location for `.md` files.
- **Incremental Export**—enabled by default. When disabled, all papers are exported each time.
- **Auto Export**—when enabled, saving notes or highlights automatically triggers an export (5-second debounce).
- **Export Now**—manually trigger a full or incremental export.

## 16.5 Annotation Color Settings

The "Settings → Annotation Colors" panel manages the mapping between highlight colors and semantic labels (see Section 4.6 for details):

- View all current color-label mappings.
- Edit label names.
- Add new color-label mappings.
- Delete unused mappings.

## 16.6 Custom Field Settings

The "Settings → Custom Fields" panel manages custom metadata field definitions for papers (see Chapter 16 for details):

- Create new fields (name + type + optional select options).
- Edit existing fields.
- Delete fields (cascading deletion of that field's values across all papers).

## 16.7 Topic Alert Settings

The "Settings → Topic Alerts" panel manages automated paper monitoring (see Section 7.4 for details):

- View all alerts and their last run times.
- View the history of results for each alert.

- Edit an alert’s query, frequency, and target folder.
- Pause or delete alerts.
- Mark results as read.

Chapter 17

# Data and File Layout

LitFolio believes "your data belongs to you." All information is stored in readable formats under `~/Litera-Library/`, so you can access it with scripts even without LitFolio.

## 17.1 Top-Level Structure

```
~/Litera-Library/
library.db                                  Main SQLite database
library.db-wal                              WAL log (runtime)
library.db-shm                              Shared memory (runtime)
papers/
<paperId>/
paper.pdf
meta.json
note.md
pdf-text.txt
notes/
ask/                                        Ask results saved as notes
backups/
deleted/                                    Paper copies before deletion
sync/                                       Local snapshots before pull
sync/                                       Temporary area for WebDAV push
```

## 17.2 library.db Table Overview

Main tables (detailed schema in the repository at `src-tauri/migrations/`, totaling 24 migration files):

### 17.2.1 Core

- `papers`—core metadata, one row per paper (id, title, authors, abstract, status, tldr, quick_read_json, translated_title, translated_abstract, bibtex, last_exported_at, ...)
- `highlights`—selection highlights and area highlights (paper_id, page, boundingRect, text, comment, color, label)
- `terms`—glossary (paper_id, term, definition, evidence, cross_refs)
- `tags` / `paper_tags`—tags and associations
- `folders` / `paper_folders`—folders and associations (supports nesting and multiple membership)

### 17.2.2 Discovery and Reading

- `feeds` / `feed_items`—RSS feeds and entries
- `topic_surveys`—saved topic surveys
- `topic_alerts` / `topic_alert_results`—topic alert configurations and results
- `reading_queue`—reading queue (paper_id, priority, target_date, note)
- `paper_comparisons`—multi-paper comparison results
- `recommendation_cache`—similar paper recommendation cache
- `paper_citations`—citation network cache

### 17.2.3 Notes and Knowledge

- `paper_note_sections`—structured note cards (paper_id, section_key, content, source, sort_order)
- `paper_links`—manual/AI relationships between papers
- `paper_terms`—paper-term associations (including cross-paper reuse)
- `concepts`—concept nodes (name, description, source)
- `concept_relations`—relationships between concepts (replaces/extends/requires/enables/competes_w
- `paper_concepts`—paper-concept associations

### 17.2.4 Configuration and Indexing

- `llm_profiles` / `task_assignments`—endpoints and task bindings
- `papers_fts` / `highlights_fts` / `terms_fts`—FTS5 full-text indexes
- `paper_embeddings`—vector embedding cache
- `custom_field_defs` / `paper_custom_fields`—custom metadata
- `smart_collections`—smart collection rules

## 17.3 papers/<id>/ Per-Paper Layout

Each paper has its own directory, named with a ULID (a 26-character string like `01KSCN65XF4B2PZ83D27ET`

**paper.pdf** The original PDF. If this file is deleted, LitFolio will display “PDF unbound,” but metadata and notes remain intact; use “Rebind PDF” in the library drawer to reattach it.

**meta.json** Metadata snapshot (title, authors, year, DOI/arXiv ID, source, etc.). SQLite is the primary storage; meta.json is a mirror for script access.

**note.md** Notes for this paper. Entirely user-written; LitFolio does not modify content beyond what you write in the reader.

**pdf-text.txt** Full-text extraction from the PDF (generated offline with pdf-extract), used for FTS5 indexing and RAG retrieval.

## 17.4 Performance Optimization

LitFolio includes two key optimizations for large-scale libraries (1000+ papers).

### 17.4.1 Virtual Scrolling

The library main list uses virtual scrolling, implemented with `@tanstack/react-virtual`. Core principles:

1. **Container Measurement**—when the list container mounts, measure the visible area height (e.g., 800px).
2. **Row Height Estimation**—estimate the height of each paper card row (e.g., 72px).
3. **Viewport Calculation**—based on scroll offset and container height, calculate the range of row indices that need to be rendered. For example, when scrolled to 500px, render rows 7–18 (plus an overscan buffer of 3 rows above and below).
4. **DOM Node Pool**—only create DOM nodes within the visible range. As the user scrolls, nodes leaving the viewport are recycled and new nodes entering the viewport are created.
5. **Placeholder Padding**—the total list height is calculated as `totalSize` × `rowHeight`, using top and bottom padding elements to maintain the scrollbar, ensuring scroll behavior is consistent with a full list.

Results:

- Initial render time $< 100$ms (2000 papers).
- Scrolling maintains 60fps—because only 20–30 DOM nodes are manipulated at a time.
- Memory usage is proportional to the visible area size, not to the total number of papers.

### 17.4.2 Embedding Cache

RAG queries (in-library asking) require vector embeddings of paper text. LitFolio uses the `paper_embeddings` table to cache generated embedding vectors:

1. Before a query, check whether the `paper_embeddings` table has a cached record for the paper and whether the `content_hash` matches the current text hash.
2. Cache hit → use the cached embedding vector directly, skipping the API call.
3. Cache miss → call the embedding API to generate the vector and write it to the cache table.
4. Paper content changes (note edits, highlight additions/deletions) → `content_hash` changes → embedding is automatically regenerated on the next query.

The embedding cache is fully transparent to users—no manual operation is required. Logs will show `"embedding cache hit"` or `"embedding cache miss"` to confirm whether the cache is working.

## 17.5 Backups

LitFolio has two types of automatic backups:

1. **Deletion Recycle Bin** (`backups/deleted/`)—as described in **Chapter 2**, deleted papers are moved here first. Automatically cleaned after 30 days.
2. **Sync Snapshots** (`backups/sync/`)—a complete local snapshot taken before "Pull and Replace." Retains the most recent 5 snapshots.

If you need to add your own daily scheduled backups, the simplest approach is to use the entire `~/Litera-Library/` as a borg/restic repository source—the SQLite WAL files are included, ensuring consistency.

Chapter 18

# Keyboard Shortcuts Quick Reference

## 18.1 Reader (/reader)

| Shortcut | Action |
|---|---|
| `Ctrl+F` / `Cmd+F` | Open PDF search box |
| `Enter` | Jump to next match |
| `Shift+Enter` | Jump to previous match |
| `Esc` | Close search box |
| `Alt+Drag` | Area highlight (rectangular screenshot) |
| `Mouse Selection` | Text highlight (bubble appears) |
| `j` / `k` | Next page / Previous page |
| `1` / `2` / `3` | Switch right panel: Notes / Selection Translation / Terms |
| `[` / `]` | Left/Right panel width 5% step |

## 18.2 Global

| Action | Effect |
|---|---|
| `Ctrl+K` / `Cmd+K` | Open command palette |
| Drag PDF to any page | Navigate to "Import → PDF File" with the file prefilled |
| `Ctrl+Enter` / `Cmd+Enter` | Send message on Ask page |
| `Esc` | Close command palette / dialog / search box |

Chapter 19

# Troubleshooting

Below are the most commonly encountered issues, organized by "Symptom → Cause → Fix."

## 19.1 LLM Call Failure

**Symptom**: TL;DR / Deep Read / Translation buttons spin and then show "Request failed."
**Common Causes and Fixes**:

1. The current profile is not configured or the model name is misspelled—go to "Settings → LLM Configuration" and click "Test Connection."
2. API key balance exhausted / authentication expired—error details are displayed as-is; refer to your provider's instructions.
3. Network unreachable (especially direct access to OpenAI from certain regions)—switch to a compatible endpoint (Azure / regional proxy).
4. Local Ollama is not running—start with `ollama serve`, then go to settings and test.

## 19.2 PDF Load Failure

**Symptom**: The reader shows a perpetual spinner or displays "Failed to load PDF."
**Fix**:

- Check whether `papers/<id>/paper.pdf` exists; if not, click "Rebind PDF" in the library drawer.
- Some scanned PDFs use non-standard encodings that cause PDF.js rendering to fail—try processing with `ocrmypdf` or `qpdf --linearize` and then rebind.

## 19.3 RSS Not Fetching / Always "Already Up to Date"

**Symptom**: The remote clearly has new content, but clicking "Refresh" shows "Already up to date."

**Cause**: LitFolio uses conditional GET (`If-Modified-Since` / `ETag`); if the remote says nothing has changed, it will not re-fetch. If you suspect the remote is reporting incorrectly, delete the feed from "Feed Management" and re-add it.

**User-Agent Blocked**: A small number of journal websites block the default UA. Enter a custom `User-Agent` header when subscribing.

## 19.4 WebDAV Sync Failure

Three common scenarios:

- **401 Unauthorized**—incorrect username or password. For Nextcloud, use an **app password** rather than your account password.
- **404 / 405**—incorrect WebDAV URL path. Nextcloud uses `/remote.php/dav/files/<user>/`, and Jianguo Cloud uses `https://dav.jianguoyun.com/dav/`.
- **Conflict / Midway Failure**—the local snapshot taken before the pull is in `backups/sync/` and can be manually restored.

## 19.5 Database Locked

**Symptom**: On startup, a "database is locked" error appears.

**Cause**: After an unexpected kill, WAL files remain. In most cases, SQLite will automatically checkpoint and recover. If LitFolio fails to start repeatedly, close all LitFolio processes and delete `library.db-wal` and `library.db-shm` (the data itself is in `library.db`; deleting the WAL only loses the most recent few seconds of unpersisted changes).

## 19.6 Partial Batch Import Failure

**Symptom**: When importing a folder, some PDFs show "Failed."

**Common Causes**:

1. **Corrupt PDF**—the file is incomplete or truncated. Verify with `qpdf --check`.
2. **No Text Layer**—scanned PDF without an OCR text layer. LitFolio will still import it (using the filename as the title), but full-text search will not cover its content. Process with `ocrmypdf` and then rebind the PDF.
3. **Permission Issues**—the PDF file is locked by another program or lacks read permissions.

## 19.7 Concept Extraction Returns Empty

**Symptom**: After clicking "Extract Concepts," no concept nodes appear in the knowledge graph.

**Common Causes**:

1. The paper has no TL;DR or deep-read results—the LLM lacks sufficient context to identify concepts. Run a quick read or deep read on the paper first.
2. The LLM returned unparseable JSON—check whether the model configured in your LLM settings supports JSON output format.
3. The paper content is too short or is not an academic paper—concept extraction works best on academic papers.

## 19.8 Citation Network / Similar Papers Empty

**Symptom**: After clicking “Citations” or “Similar Papers,” it shows “No data available.”

**Cause**:

- The paper has no DOI and its title cannot be matched to a record in Semantic Scholar.
- The paper is very new (published less than 1 week ago) and has not yet been indexed by S2.
- Niche field or non-English paper—S2 coverage is limited in some areas.
- Network issues—check whether `api.semanticscholar.org` is accessible.

## 19.9 Smart Collection Not Updating

**Symptom**: After changing a paper’s tags or status, the smart collection’s paper list does not change.

**Cause**: Smart collections are queried in real time—you need to re-select the collection to see the update. This is not a bug but a design choice: avoiding rule queries triggered by every paper change to prevent performance impacts.

Chapter A

# LLM Endpoint Compatibility List

LitFolio follows the "OpenAI Chat Completions protocol." The following endpoints have been verified to work directly:

| Service | API URL | Notes |
|---|---|---|
| OpenAI Official | `https://api.openai.com/v1` | Direct access may be restricted |
| DeepSeek | `https://api.deepseek.com/v1` | Recommended: great value |
| Azure OpenAI | `<resource>.openai.azure.com/openai/deployments/<dep>/v1` | Requires API version header |
| Ollama (local) | `http://localhost:11434/v1` | Fully offline |
| LM Studio (local) | `http://localhost:1234/v1` | Desktop GUI loads GGUF |
| OpenRouter | `https://openrouter.ai/api/v1` | One key, multiple models |
| Together.ai | `https://api.together.xyz/v1` | Open-source model hosting |

**Task Assignment Recommendations** (specific models change rapidly; follow the approach and pick what is currently best):

- **Student / Fully Free**—run everything possible locally (local Ollama with 7B–14B open-source models is sufficient for TL;DR / translation / search expansion). Reserve online calls for "deep read," "survey," and "ask," using free tiers from domestic providers.
- **Researcher / Best Value**—point all tasks to the **budget tier of a major domestic API** (DeepSeek, Doubao, Zhipu, Moonshot—pick one). Low unit price, large context window, sufficient for the vast majority of research scenarios.
- **Power User / No Compromise**—assign TL;DR / translation / terms to a domestic budget tier; assign deep read / survey / ask to the **strongest flagship chat model available at the time** (OpenAI, Anthropic, Google take turns leading—pick one based on release date) to guarantee the highest quality answers.

Chapter B

# arXiv Main Category Quick Reference

**cs.AI / cs.LG / cs.CL** Artificial Intelligence / Machine Learning / Natural Language Processing

**cs.CV** Computer Vision

**cs.CR / cs.DC** Cryptography and Security / Distributed Systems

**stat.ML** Statistical Machine Learning

**physics.optics** Optics and Photonics

**physics.atom-ph** Atomic Physics

**cond-mat.mes-hall** Condensed Matter – Mesoscopic and Nanoscale

**quant-ph** Quantum Physics

**math.OC** Optimization and Control

**math.AP** Partial Differential Equations

**q-bio.NC** Neuroscience

For the complete list, see `https://arxiv.org/category_taxonomy`.

Chapter C

# Recommended RSS Feeds

**arXiv Subcategories** (replace `CAT` with e.g. `cs.LG`):

- `http://export.arxiv.org/rss/CAT`—official RSS, refreshed daily
- `http://export.arxiv.org/rss/CAT.recent`—latest only

**Journals**:

- Nature: `https://www.nature.com/nature.rss`
- Science: `https://www.science.org/rss/news_current.xml`
- Nature Photonics: `https://www.nature.com/nphoton.rss`
- Optica (OSA): `https://www.optica-opn.org/rss/news.xml`
- PRL: `http://feeds.aps.org/rss/recent/prl.xml`

**Research Blogs / Surveys**:

- Distill: `https://distill.pub/rss.xml` (no longer updated, but archived articles remain valuable)
- Lil'Log: `https://lilianweng.github.io/index.xml`

Chapter D

# License

LitFolio is released under the **GPL-3.0-or-later** license. This means:

- You are free to use, modify, and redistribute.
- If you create and distribute a derivative work based on LitFolio, the derivative must also be released under the same license (or a compatible more permissive license) and include source code.
- No warranty of any kind, express or implied, is provided.

The full license text is in the `LICENSE` file in the repository root directory.